Representation Learning
Autoencoders and Disentanglement

Loïc Le Folgoc
LTCI, Télécom Paris, IP Paris
loic.lefolgoc@telecom-paris.fr

Introduction

- Modern auto-encoders can **generate highly realistic images**

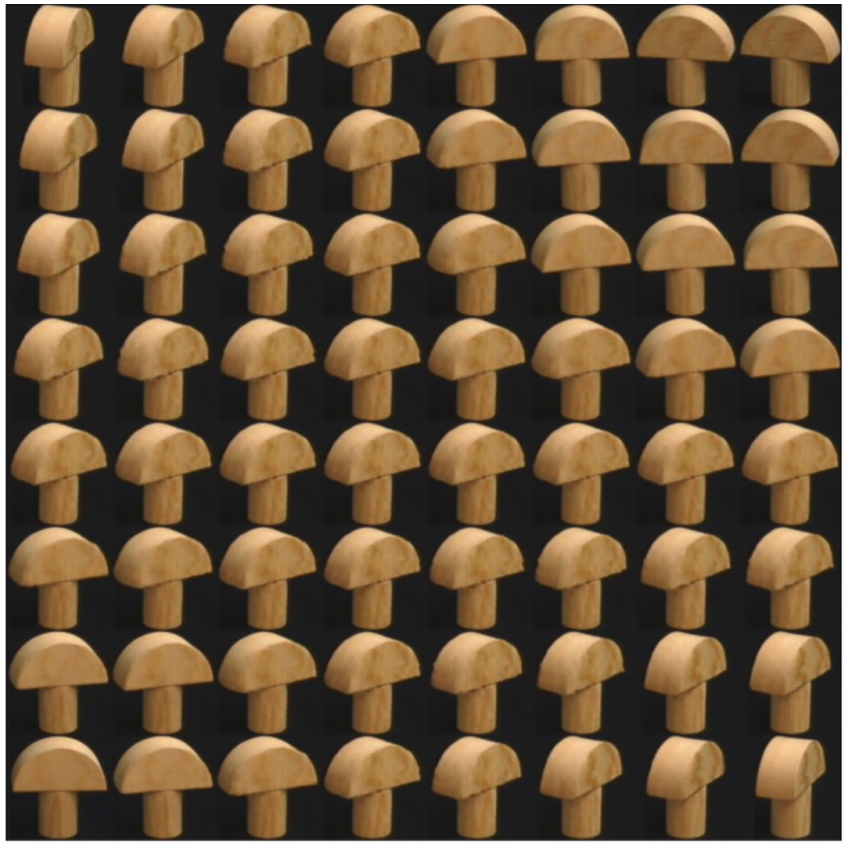




[Vahdat, 2020] **NVAE: A deep hierarchical variational autoencoder**, Vahdat, A. et Kautz, J., *NeuRIPS*, *2020*
[Pidhorskyi, 2020] **Adversarial Latent Autoencoders**, Pidhorskyi, S. et al., *CVPR*, *2020*

Introduction

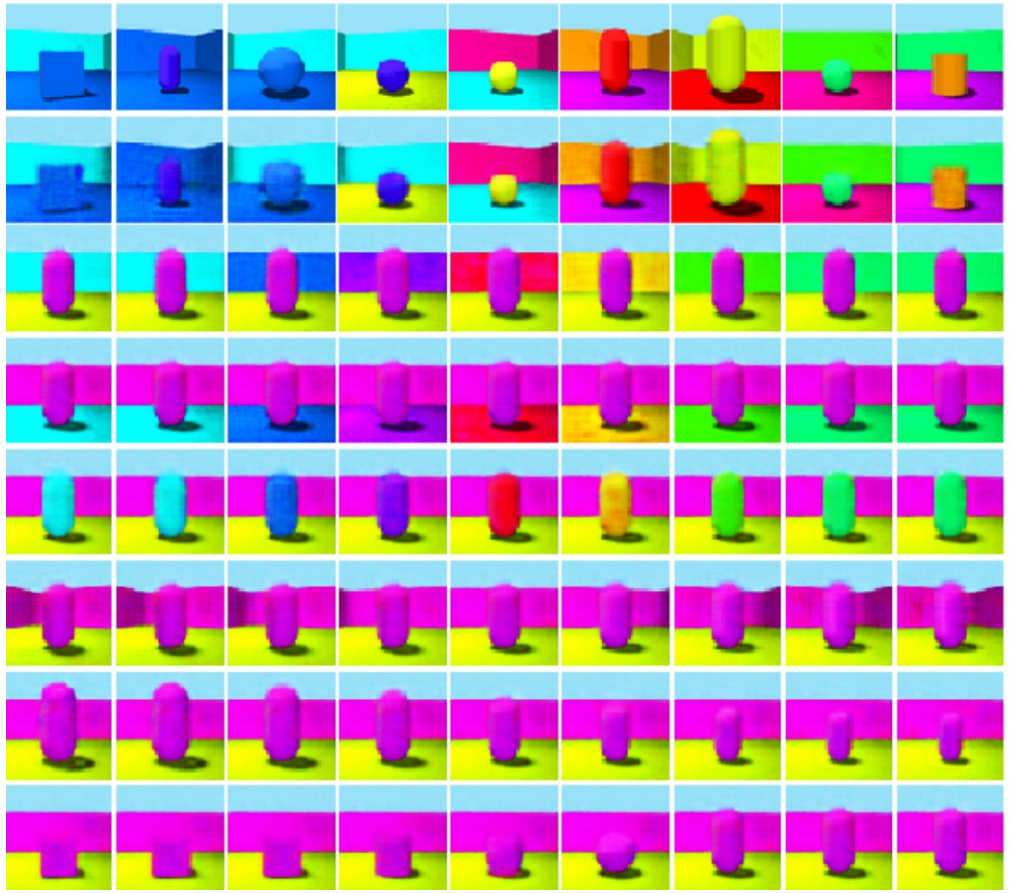
- They have other advantages, such as enabling **latent space interpolation**



[Oring, 2021] **Autoencoder Image Interpolation by Shaping the Latent Space**, Oring, A. et al., *ICML*, *2021*

Introduction

- They can be trained to favor **disentanglement of generative factors**



[Kim, 2018] **Disentangling by factorizing**, Kim, H. et Mnih, A., *ICML*, *2018*

Introduction

- VAEs are **probabilistic models**, which integrate naturally in probabilistic tasks:
  - for probabilistic segmentation* (below, of noisy faces)
  - as shape models / priors** in medical imaging
  - etc.



*[Sohn, 2015] **Learning structured output representation using deep conditional generative models**, Sohn, K. et al., *NIPS*, *2015*
[Dalca, 2019] **Anatomical priors in convolutional networks for unsupervised biomedical segmentation, Dalca, A. V. et al., *CVPR*, *2018*

Introduction

- AEs and **diffusion models** are closely related, and can be combined in various ways in the SOTA

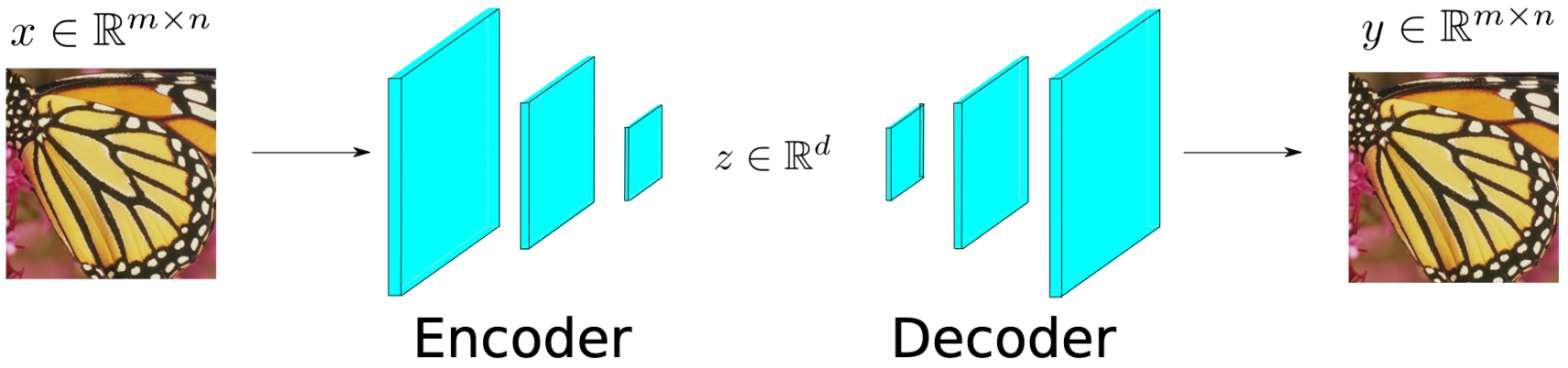


[Preechakul, 2022] **Diffusion Autoencoders: Toward a Meaningful and Decodable Representation**, Preechakul, K. et al., *CVPR*, *2022*

Auto-Encoders

Auto-Encoders

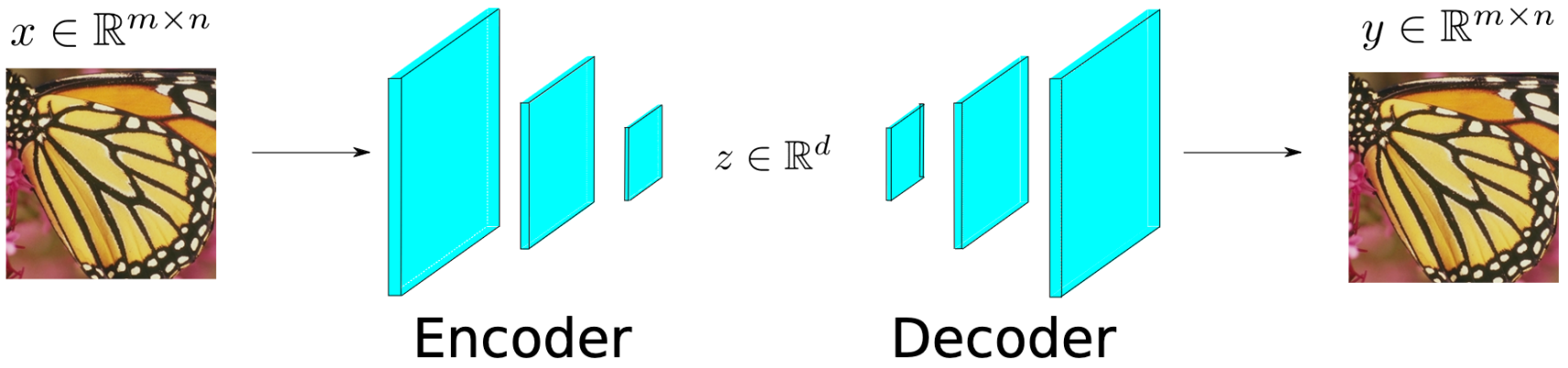
- Auto-encoders are **encoder-decoder** architectures generally **trained to reconstruct** their input
- The input passes through a bottleneck $z \in \mathbb{R}^d$, $d \ll m \times n$, which forces the auto-encoder to learn a compact and powerful **latent code** (z)



$$\mathcal{L} \triangleq \mathbb{E}_{x \sim \mathbb{P}_x}[\|y - x\|_2^2]$$

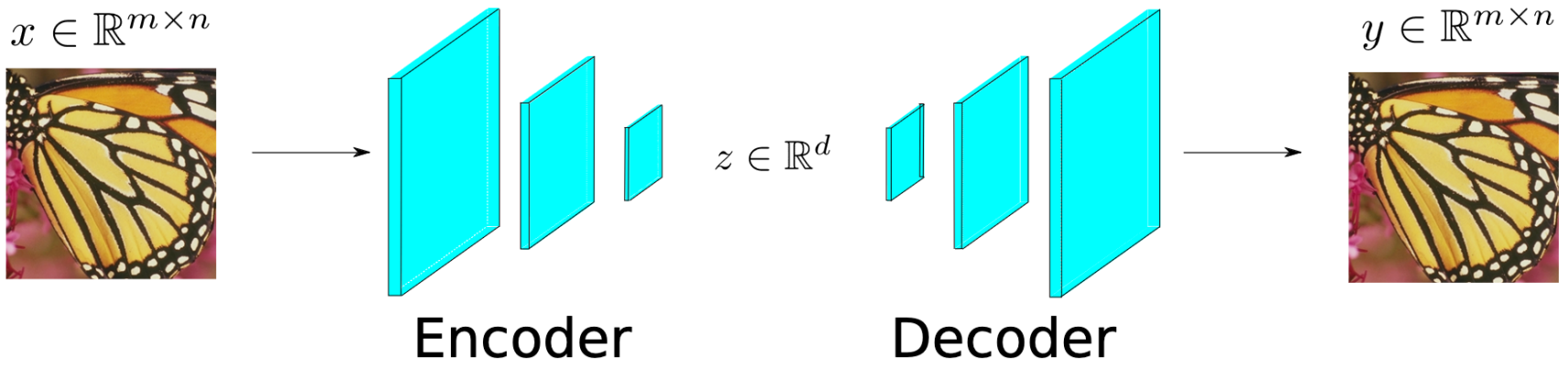
Uses of Auto-Encoders

- Data compression, dimensionality reduction
- Classification (easier in latent space)
- Data generation/synthesis



Uses of Auto-Encoders

- Data compression, dimensionality reduction
- Classification (easier in latent space)
- **Data generation/synthesis**



Is it easy to **generate data** using an auto-encoder ?

The encoder and the decoder

- An AE is a neural network consisting of two sub-networks
  - The encoder Φ_e,

$$\Phi_e: \mathbb{R}^{mn} \to \mathbb{R}^d$$
$$x \mapsto \Phi_e(x) = z$$

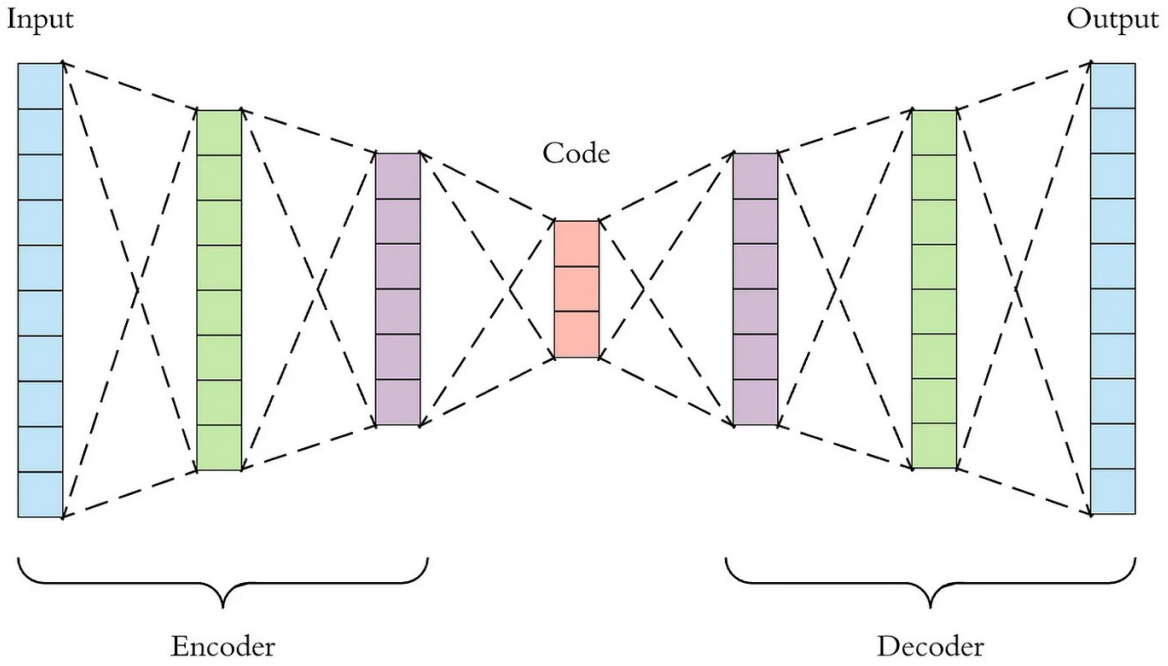
  - The decoder Φ_d,

$$\Phi_d: \mathbb{R}^{d} \to \mathbb{R}^{mn}$$
$$z \mapsto \Phi_d(z) = y$$

- Note, PCA gives the **optimal linear auto-encoder** in the sense of squared $L2$-norm reconstruction, with $\Phi_e(x) \coloneqq W^T x$ and $\Phi_d(z) = Wz$
  - W is an $(mn)\times d$ matrix whose columns are the leading eigenvectors (of largest eigenvalues) of the covariance matrix of the training data

Auto-encoder architectures

- The main components of AEs are:
  - MLPs / fully-connected layers or;
  - Convolutional neural networks (convolutions, biases and non-linearities)



Fully-Connected AE

Image credit: Arden Dertat, Towards Data Science

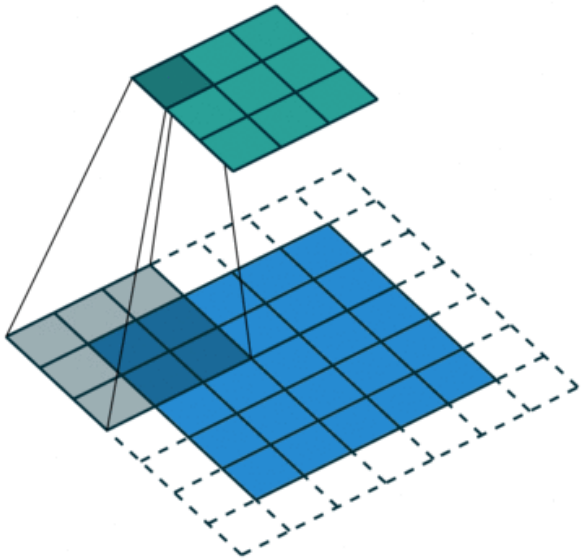
Convolutional AEs – Downsampling

- How do we create the bottleneck: **downsampling**

- We know how to downsample from introductory courses to Deep Learning:
  - Strided convolutions, or
  - Max pooling

- How about **upsampling**? (for the decoder)

Convolution with stride 2:

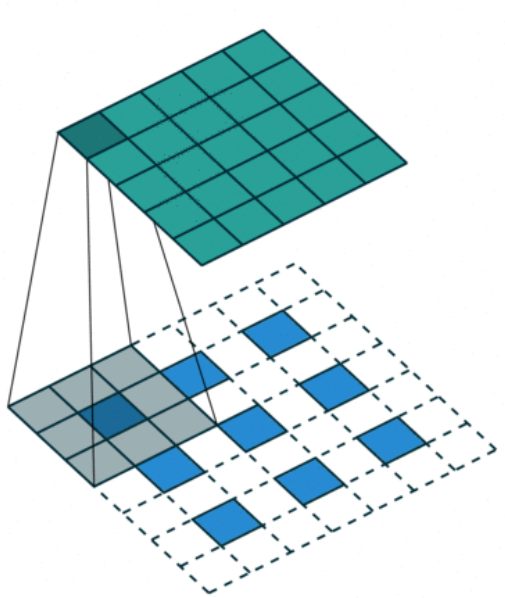


Animation: github.com/vdumoulin

Convolutional AEs – Upsampling

- Upsampling can be carried out in several ways
  - Simple interpolation (linear, bilinear, bicubic)
    `torch.nn.Upsample`
  - Transposed convolution
    `torch.nn.ConvTranspose2d`
- **Transposed convolution** is probably the most common upsampling
  - Simultaneously upsamples and convolves
- Transposed convolutions are also what happens in the backward pass of a convolutional layer!
  - Remember that $\frac{\partial \mathcal{L}}{\partial x}$ can also be expressed as a convolution!

Transposed convolution with stride 2:



Animation: github.com/vdumoulin

Transposed Convolution

- Recall that we can write the convolution as a matrix/vector multiplication
- For example, take the convolution with the Laplacian operator

$$w = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix} \rightarrow A_w = \begin{pmatrix} 4 & -1 & \overset{0}{\cdots} & -1 & \overset{0}{\cdots} & & \\ -1 & 4 & -1 & \overset{0}{\cdots} & -1 & \overset{0}{\cdots} & \\ 0 & -1 & 4 & -1 & \overset{0}{\cdots} & -1 & \overset{0}{\cdots} \\ \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \\ -1 & & \overset{0}{\cdots} & -1 & \overset{0}{\cdots} & -1 & 4 \end{pmatrix}$$

- To carry out strided convolutions (= convolution + subsampling) with stride s, we just remove certain rows from the matrix A_w (the elements not retained during subsampling)

Transposed Convolution

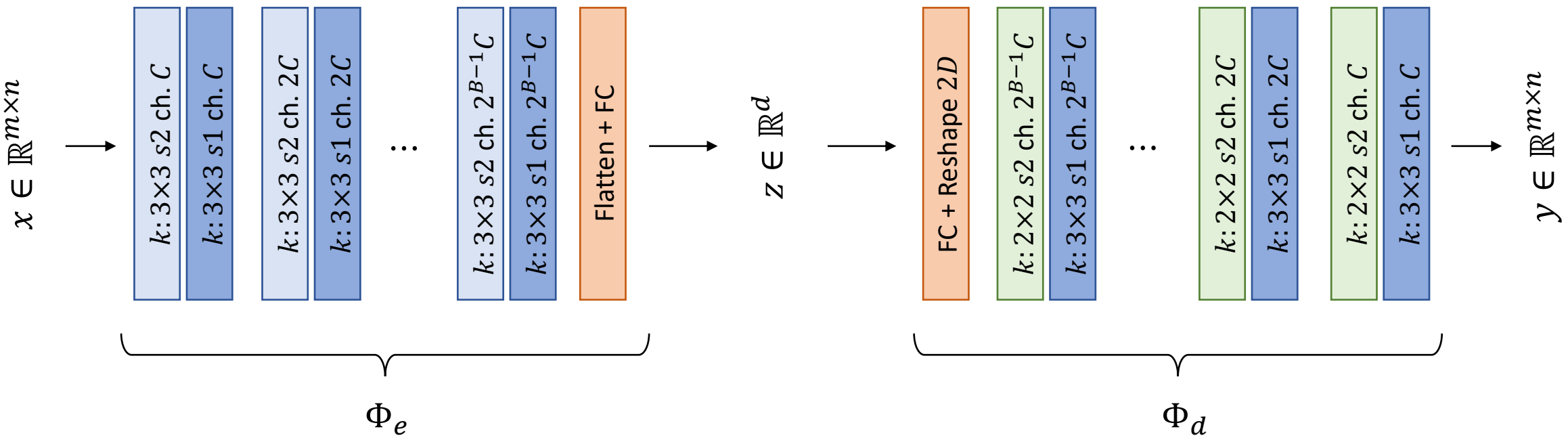
- To carry out strided convolutions (= convolution + subsampling) with stride s, we just remove certain rows from the matrix A_w (the elements not retained during subsampling)

$$A_{w,s} \triangleq \begin{pmatrix} 4 & -1 & \overset{0}{\cdots} & -1 & \overset{0}{\cdots} & & \\ -1 & 4 & -1 & \overset{0}{\cdots} & -1 & \overset{0}{\cdots} & \\ 0 & -1 & 4 & -1 & \overset{0}{\cdots} & -1 & \overset{0}{\cdots} \\ \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \\ -1 & & \overset{0}{\cdots} & -1 & \overset{0}{\cdots} & -1 & 4 \end{pmatrix}$$

- A_w now becomes an $\frac{mn}{s^2} \times mn$ matrix $A_{w,s}$
- **Transposed convolution** just consists of the $mn \times \frac{mn}{s^2}$ matrix $A_{w,s}^T$

Convolutional Auto-Encoders

- An example of convolutional AE architecture with B blocks in the encoder and decoder (e.g., $C := 32$):



k: $(k \times k)$ $s(s)$ ch. (C)	Convolution with kernel $k \times k$ and stride s, outputting C feature maps
k: $(k \times k)$ $s(s)$ ch. (C)	Transposed convolution with kernel $k \times k$ and stride s, outputting C feature maps

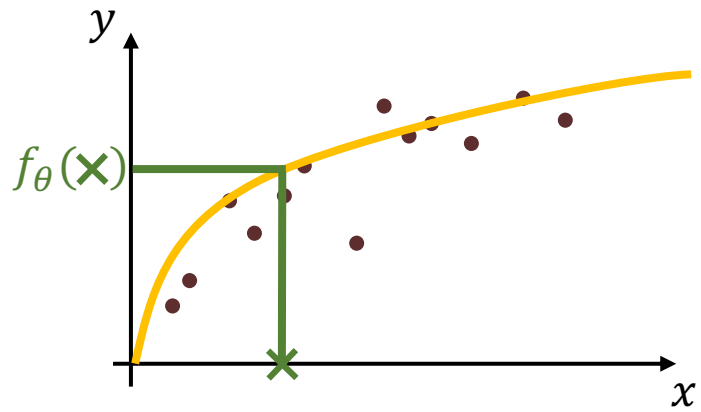
Variational Auto-Encoders

Probabilistic models

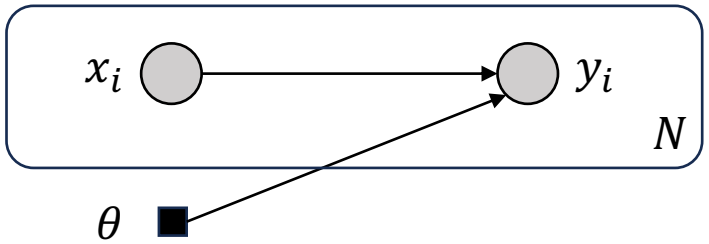
- Probabilistic models are hidden behind most **energy minimization** problems in machine learning
- Example of **Ordinary Least Squares** regression (OLS): fit a regressor $\hat{y} = f_\theta(x)$, $\theta \in \Theta$, to training data $\{x_i, y_i\}_{i=1\cdots N}$ using the following $L2$-loss, where $x_i \in \mathcal{X}$ and $y_i \in \mathbb{R}^d$:

$$\mathcal{L}(\theta) \triangleq \sum_{i=1}^{N} \|y_i - f_\theta(x_i)\|^2$$

- This is also known as the Sum of Squared Differences (SSD) or Mean Squared Error (MSE)



Probabilistic models



- Alternatively, let's **make explicit** how the **data is generated**: define a **data likelihood**

$$p_\theta(y|x) \triangleq \mathcal{N}(y|f_\theta(x), \sigma^2 \mathrm{I}_d) = \frac{1}{\left(\sigma\sqrt{2\pi}\right)^d} \exp\left(-\frac{1}{2\sigma^2}\|y - f_\theta(x)\|^2\right)$$

- Further assume that the data $\mathcal{D} = \{X, Y\} = \{x_i, y_i\}_{i=1\ldots N}$ is i.i.d.: $p_\theta(Y|X) = \prod_{i=1}^N p_\theta(y_i|x_i)$

- The likelihood $p_\theta(Y|X)$ tells us how probable the data is under the model $\theta \in \Theta$
- Computation shows that **Maximum likelihood estimation** (MLE) of θ is equivalent to OLS regression:

$$\underset{\theta}{\operatorname{argmax}} \log p_\theta(Y|X) = \underset{\theta}{\operatorname{argmin}} \frac{1}{2\sigma^2} \sum_{i=1}^N \|y_i - f_\theta(x_i)\|^2 = \underset{\theta}{\operatorname{argmin}} \mathcal{L}(\theta)$$

Probabilistic models

- Let's add a prior over $\theta \in \mathbb{R}^K$:

$$p(\theta) \triangleq \mathcal{N}\left(\theta \middle| 0, \frac{1}{\gamma} \mathrm{I}_K\right) = \left(\frac{\gamma}{2\pi}\right)^{\frac{K}{2}} \exp\left(-\frac{\gamma}{2}\|\theta\|^2\right)$$

- Bayes' rule tells us how to invert the likelihood to obtain the posterior distribution of θ:

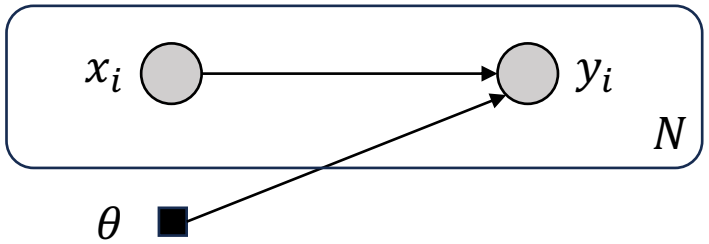
$$p(\theta|Y,X) = \frac{p(Y|X,\theta)p(\theta)}{p(Y|X)}$$

- With **Maximum A Posteriori** (MAP) estimation, we recover Ridge Regression
(note that $\underset{\theta}{\operatorname{argmax}}\, p(\theta|Y,X) = \underset{\theta}{\operatorname{argmin}} -\log p(\theta|Y,X)$)

$$-\log p(\theta|Y,X) = -\log p(Y|X,\theta) - \log p(\theta) + \text{cst.}$$

$$= \frac{1}{2\sigma^2}\sum_{i=1}^{N}\|y_i - f_\theta(x_i)\|^2 + \frac{\gamma}{2}\|\theta\|^2 + \text{cst.}$$

Probabilistic models



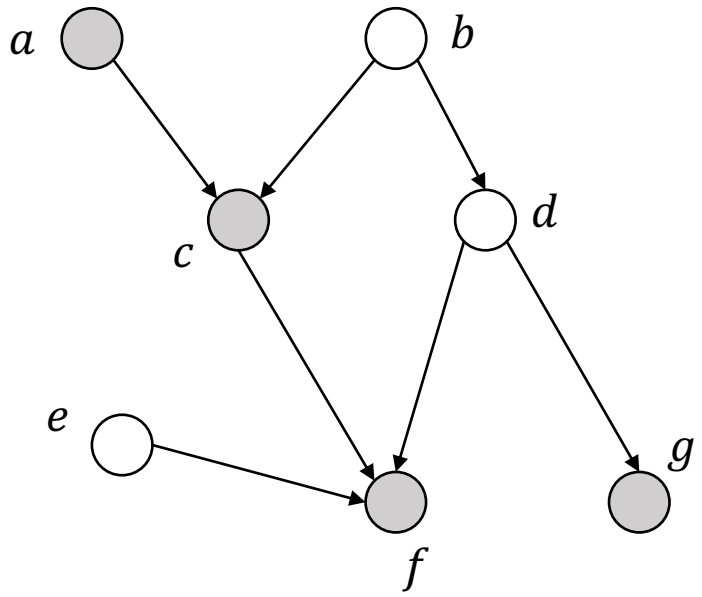
- Similarly, we can derive the **categorical cross-entropy** loss using a categorical distribution as the data likelihood and an i.i.d. assumption:

$$p(y|x;\theta) = \mathcal{C}(y|\sigma_\theta(x)) = \prod_{c=1}^{C} (\sigma_\theta(x)_c)^{y_c}$$

- Here, y is a one-hot encoded label/class and $\sigma_\theta(x)$ a C-way predicted probability vector for input x (this reads as: class c has probability $\sigma_\theta(x)_c$)

- MLE maximizes $\log p(Y|X;\theta) = \sum_{i=1}^{N} \sum_{c=1}^{C} y_{ic} \log \sigma_\theta(x_i)_c$, which is minus the cross-entropy loss

Graphical models



$$p(a,b,c,d,e,f,g) = p(a)p(b)p(e)p(c|a,b)p(d|b)$$
$$p(f|e,c,d)p(g|d)$$

- Graphical models are powerful representations of our assumptions of **how the joint distribution factorizes**, without having to make explicit the choice of probability distributions for each factor
- In general, $p(v_1, \dots, v_n) = \prod_{i=1\dots n} p(v_i|\mathbf{pa}(v_i))$, with $\mathbf{pa}(v_i)$ the parent nodes of v_i
- Algorithms can be designed for inference exploiting the graph structure

Probabilistic, Bayesian and Structural Causal Models

- Probabilistic and Bayesian modeling go way beyond the focus of this course; you can find comprehensive references on these topics:

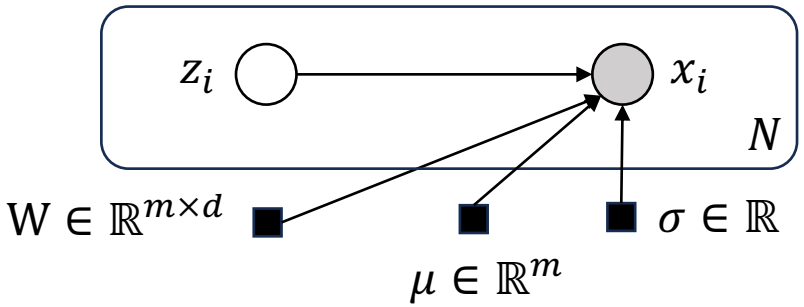
  - Barber, David. *Bayesian reasoning and machine learning*. Cambridge University Press, 2012.
  - Koller, Daphne, and Nir Friedman. *Probabilistic graphical models: principles and techniques*. MIT press, 2009.
  - Bishop, Christopher M., and Nasser M. Nasrabadi. *Pattern recognition and machine learning*. Springer, 2006.
  - Jordan, Michael I. "An introduction to probabilistic graphical models." Unfinished book, 2003.
  - Berger, James O. *Statistical decision theory and Bayesian analysis*. Springer Science & Business Media, 2013.

- Probabilistic models can be extended to a **theory of causality**; with the possibility to evaluate the result of **interventions** (what will happen if I do X?) thanks to the **do-calculus** and to evaluate **counterfactuals** (what would have happened, had I done X'; given that Z happened when I did X?)

  - Pearl, Judea. *Causality*. Cambridge university press, 2009.
  - Spirtes, Peter, Clark N. Glymour, and Richard Scheines. *Causation, prediction, and search*. MIT press, 2000.
  - Peters, Jonas, Dominik Janzing, and Bernhard Schölkopf. *Elements of causal inference: foundations and learning algorithms*. The MIT Press, 2017.

Probabilistic PCA

- There is a simple linear-Gaussian **generative model** that algorithmically **leads to PCA**

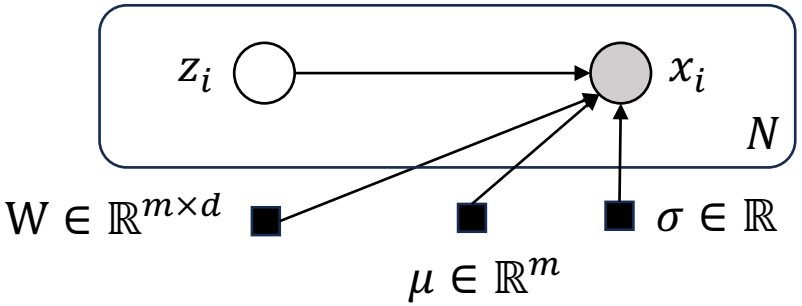


- Assume that $x_i \in \mathbb{R}^m$ is generated linearly from a set of independent variables $z_i \in \mathbb{R}^d$, up to some independent noise (we note $\theta \triangleq \{\mathrm{W}, \mu, \sigma\}$ the model parameters):

$$p_\theta(x|z) \triangleq \mathcal{N}(x \mid \mathrm{W}z + \mu, \sigma^2 \mathrm{I}_m)$$
$$p(z) \triangleq \mathcal{N}(z \mid 0, \mathrm{I}_d)$$

- The probabilistic model above is fully specified (the joint distribution is uniquely defined) as all the factors $p(v_i|\mathbf{pa}(v_i))$ have been specified

Probabilistic PCA: Learning the model

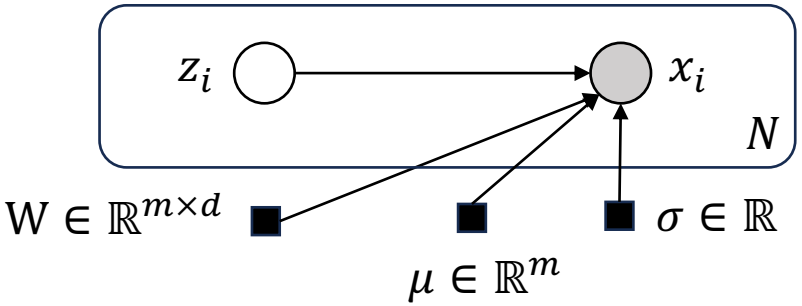


$$p_\theta(x|z) \triangleq \mathcal{N}(x \mid \mathrm{W}z + \mu, \sigma^2 \mathrm{I}_m)$$
$$p(z) \triangleq \mathcal{N}(z \mid 0, \mathrm{I}_d)$$

- This is a **latent variable** model (z_i's are not observed), whose non-linear generalization is VAEs
- How do we infer the optimal hyperparameters $\theta \in \Theta$ from data $X = \{x_i\}_{i=1\ldots N}$?
- We maximize the probability $p_\theta(X)$ of the observed data X under the model θ; this quantity is known as the **marginal likelihood** (or *evidence*) because it is obtained by marginalizing over the latent variables Z:

$$\begin{aligned} p_\theta(X) &= \int p_\theta(X|Z)p(Z)dZ \\ &= \iiint \prod_i \big(p_\theta(x_i|z_i)p(z_i)\big)dz_1 \cdots dz_n \\ &= \prod_i \int p_\theta(x_i|z_i)p(z_i)dz_i = \prod_i p_\theta(x_i) \end{aligned}$$

Probabilistic PCA: Learning the model



$$p_\theta(x|z) \triangleq \mathcal{N}(x \mid \mathrm{W}z + \mu, \sigma^2 \mathrm{I}_m)$$
$$p(z) \triangleq \mathcal{N}(z \mid 0, \mathrm{I}_d)$$

- For this model, $p_\theta(x_i)$ is available in closed form: $p_\theta(x_i) = \mathcal{N}(x_i \mid \mu, \mathrm{C})$, with $\mathrm{C} \triangleq WW^T + \sigma^2 \mathrm{I}_m$
- We can minimize analytically $-\log p_\theta(X) = -\sum_i \log p_\theta(x_i)$ w.r.t. θ; tedious derivations* give :

$$\mu := \frac{1}{N}\sum_i x_i$$

$$W := \mathrm{U}_d(\Lambda_d - \sigma^2 \mathrm{I}_d)^{1/2}\mathrm{R}$$

$$\sigma^2 := \frac{1}{m-d}\sum_{j=d+1}^{m} \lambda_j$$

Where,

- $\mathrm{U}_d \in \mathbb{R}^{m \times d}$ has for columns the d largest eigenvectors of the covariance matrix of X, $\Lambda_d \triangleq \mathrm{diag}(\lambda_1, \cdots, \lambda_d)$ its d largest eigenvalues
- R is an arbitrary $d \times d$ orthogonal rotation matrix

*[Tipping, 1999] **Probabilistic Principal Component Analysis**, Tipping, M. E. et Bishop, C. M., *Journal of the Royal Statistical Society Series B: Statistical Methodology,* 1999

Probabilistic PCA: Encoding

- In practice, PPCA is often used for purposes of **data compression**: we are going to summarize a new x_* by its "corresponding" latent code z_*

- In fact, we have a whole distribution of latent codes corresponding to x_*, given by the posterior distribution:

$$p_\theta(z_*|x_*) = \frac{p_\theta(x_*|z_*)p(z_*)}{p(x_*)}$$

- The posterior is also available in closed form* and Gaussian: $p_\theta(z_*|x_*) = \mathcal{N}(z_* \mid z_\mu^*, \sigma^2 \mathrm{M}^{-1})$, with $z_\mu^* \triangleq \mathrm{M}^{-1}\mathrm{W}^T(x_* - \mu)$ and $\mathrm{M} \triangleq \mathrm{W}^T\mathrm{W} + \sigma^2 \mathrm{I}_d$

- We can thus **encode** x_* by the **posterior mean** z_μ^*

*[Tipping, 1999] **Probabilistic Principal Component Analysis**, Tipping, M. E. et Bishop, C. M., *Journal of the Royal Statistical Society Series B: Statistical Methodology,* 1999

Probabilistic PCA: Decoding

- How to **decode** z_μ^* into $\hat{x}_* \approx x_*$?
  - We can use the mean of the likelihood $p\left(x \middle| z_\mu^*\right)$ i.e., $\hat{x}_* \coloneqq \mathbb{E}\left[x \middle| z_\mu^*\right] = W z_\mu^* + \mu$

- The encoding-decoding pipeline reveals a linear **auto-encoder**:

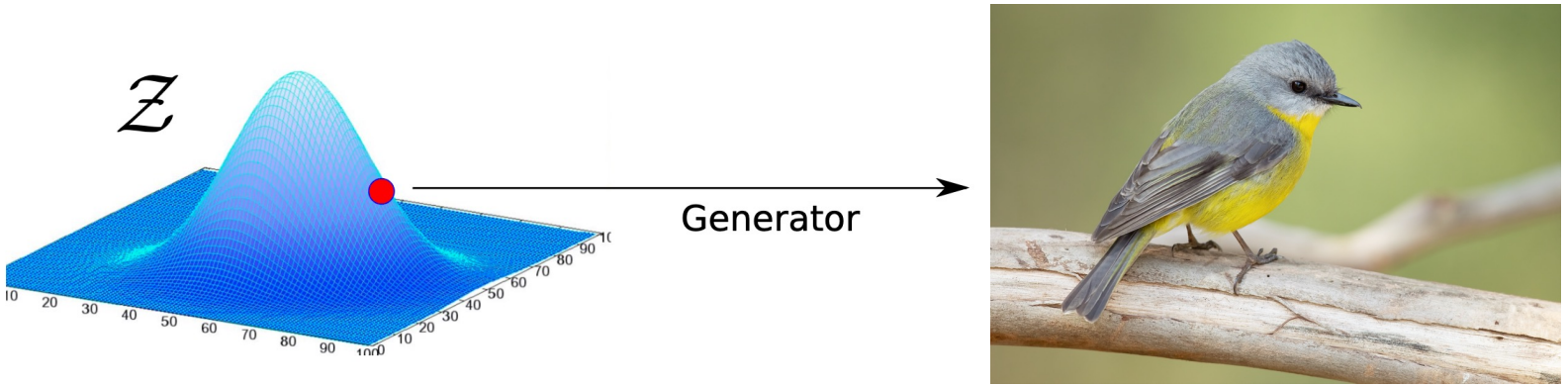
$$x_* \in \mathbb{R}^m \xrightarrow[p_\theta(z_*|x_*)]{\text{Encoder}} z_\mu^* \in \mathbb{R}^d \xrightarrow[p_\theta\left(x \middle| z_\mu^*\right)]{\text{Decoder}} \hat{x}_* \in \mathbb{R}^m$$

- Note: for this linear-Gaussian model, we can do slightly better reconstruction than $\mathbb{E}\left[x \middle| z_\mu^*\right]$ (in the sense of minimizing the expected squared reconstruction error), but this is of no interest to us today (cf. [Tipping, 1999] for the formula)

*[Tipping, 1999] **Probabilistic Principal Component Analysis**, Tipping, M. E. et Bishop, C. M., *Journal of the Royal Statistical Society Series B: Statistical Methodology,* 1999

VAEs

- The VAE is a generalization of the PPCA **latent probabilistic model** which (in general) uses instead a **non-linear generative model**, usually involving a **neural network**



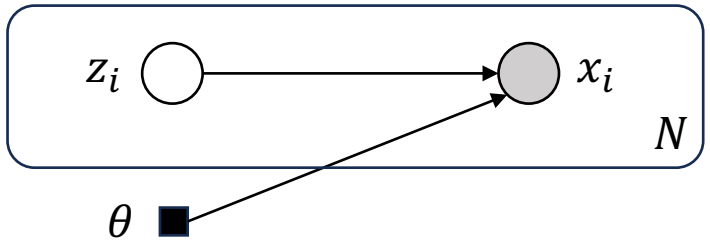
Latent space

Synthesis of random image

- Compared to non-probabilistic auto-encoders, it is much **easier to sample from** as it is generative:
  1. Sample $z_* \sim p(z)$
  2. Decode z_* i.e., sample $x_* \sim p_\theta(x|z_*)$

VAE model

- The corresponding graphical/generative model is basically unchanged, only the specifics of the probability distributions change



- The most common choice is for the **prior** $p(z) \triangleq \mathcal{N}(z|\, 0, \mathrm{I}_d)$ to be **Gaussian**

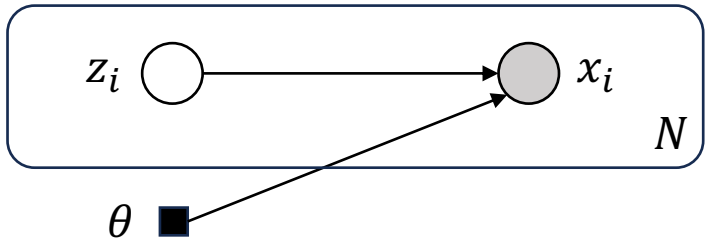
- For **natural** color ($c \coloneqq 3$) **images** or greyscale ($c \coloneqq 1$) medical images $x \in \mathbb{R}^{m \times n \times c}$, the **likelihood** $p_\theta(x|z) \triangleq \mathcal{N}(x|\, \mu(z), \tau^2 \mathrm{I}_{m \times n \times c})$ is generally **Gaussian** with i.i.d. noise over every pixel; the mean $\mu(z)$ is given by a **non-linear mapping** such as a neural network g_θ with parameters θ:

$$\mu(z) \triangleq g_\theta(\mathrm{z}) \in \mathbb{R}^{m \times n \times c}$$

- This model is called a **Gaussian VAE**

VAE model

- The corresponding graphical/generative model is basically unchanged, only the specifics of the probability distributions change



- The most common choice is for the **prior** $p(z) \triangleq \mathcal{N}(z|\, 0, \mathrm{I}_d)$ to be **Gaussian**

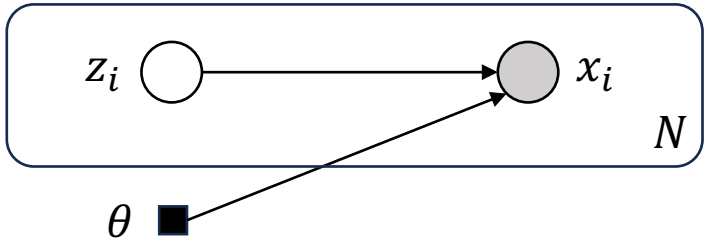
- For **multiclass** segmentation **masks** $x \in \{1, \cdots, K\}^{m \times n}$ (integer encoding) or $x \in \{0,1\}^{m \times n \times K}$ (one-hot encoding), the likelihood $p_\theta(x|z)$ is generally a product of independent **categorical distributions** $\mathcal{C}(x^{(p)}|g_\theta(z)^{(p)})$ over every pixel $p = 1, \ldots, mn$:

$$p_\theta(x|z) \triangleq \prod_{p=1 \cdots mn} \mathcal{C}(x^{(p)}|g_\theta(z)^{(p)})$$

- $g_\theta(z) \in [0,1]^{m \times n \times K}$ is a probability map output by a NN g_θ with parameters θ (e.g., via a softmax layer)

VAE: Learning the model parameters?

- How to fit the parameters θ from a dataset of images $X = \{x_i\}_{i=1\ldots N}$?



- As before, we wish to **maximize the marginal likelihood** $p_\theta(X) = \prod_i p_\theta(x_i)$

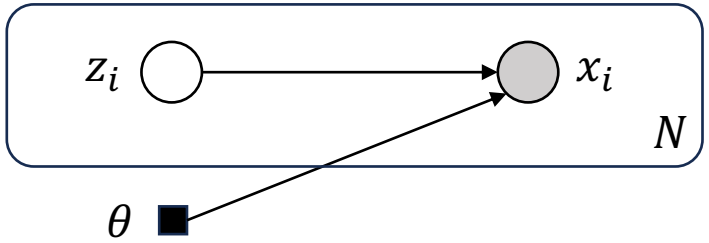
- **Problem**: this integral is **intractable** because of the NN non-linearity g_θ inside the data likelihood $p_\theta(x_i|z_i)$:

$$p_\theta(x_i) = \int p_\theta(x_i|z_i)p(z_i)dz_i$$

→ Unlike for PPCA, we cannot infer θ in closed form

VAE: Encoding data points?

- We also want to find latent codes z_* that best encode an image x_*



- As before, we turn to the posterior distribution $p_\theta(z_*|x_*)$; Bayes' rule gives as usual:

$$p_\theta(z_*|x_*) = \frac{p_\theta(x_*|z_*)p(z_*)}{p(x_*)}$$

- **Problem**: this distribution is **intractable** because of the NN non-linearity g_θ inside the data likelihood $p_\theta(x_*|z_*)$

→ Unlike for PPCA, we cannot encode x_* in closed form

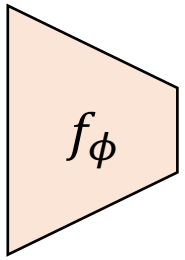
Variational Bayesian inference

- We introduce a learnable parametric approximation $q_\phi(z|x)$ of the true posterior $p_\theta(z|x)$, called the **variational posterior**
- $q_\phi(z|x)$ will consist of a **neural network** with parameters ϕ that outputs a distribution for any input x; for this reason, it will also be called **the inference network**
- Most commonly, the inference network predicts the parameters of a diagonal Gaussian distribution:

$$q_\phi(z|x) \triangleq \mathcal{N}\left(z \mid \mu(x;\phi), \mathrm{diag}\left(\sigma(x;\phi)\right)^2\right)$$



$x \in \mathbb{R}^{m \times n \times c}$



$\mu(x) \in \mathbb{R}^d$

$\log_\sigma(x) \in \mathbb{R}^d$

Variational Bayesian inference

- It turns out we can use the variational posterior q_ϕ to construct a **tractable lower-bound** $\mathcal{L}(\theta, \phi, x)$ of the log-marginal likelihood $\log p_\theta(x)$ that we will numerically maximize

- The quality of the lower-bound $\mathcal{L}(\theta, \phi, x)$ depends on the choice of q_ϕ: in fact, the q_ϕ that leads to the **best lower-bound** (smallest gap) also provides the **best posterior approximation**!

$$\underbrace{\log p_\theta(x)}_{\text{large if the data } x \text{ is likely under the model } \theta} = \mathcal{L}(\theta, \phi, x) + \underbrace{KL\left[q_\phi(\cdot\,|x) \| p_\theta(\cdot\,|x)\right]}_{\text{small if } q_\phi(z|x) \text{ approximates } p_\theta(z|x) \text{ well}} \tag{1}$$

- Optimizing $\mathcal{L}(\theta, \phi)$ jointly w.r.t. θ, ϕ solves both problems of learning the optimal θ and of encoding data points x optimally!

- $\mathcal{L}(\theta, \phi, x)$ is called the **Evidence Lower Bound** (ELBO)

Kullbach-Leibler divergence

Kullback-Leibler divergence

Let p and q be two probability distributions defined over the same domain. The Kullback-Leibler divergence is defined as

$$KL(q \,||\, p) = \int q(x) \log \frac{q(x)}{p(x)} dx$$

The Kullback-Leibler divergence has some interesting properties :

- Non-negative : $KL(q \,||\, p) \geq 0$
- $KL(q \,||\, p) = 0 \iff q = p$ almost everywhere
- Non-symmetric : $KL(p \,||\, q) \neq KL(q \,||\, p)$

The second point is quite important. Why ? Because we know that **by minimising the KL divergence we are necessarily forcing p and q closer together**.

Credit: Alasdair Newson

Variational Bayesian Inference

- As announced, the ELBO $\mathcal{L}(\theta, \phi, x)$ is tractable; taking Eq. (1) as definition, we can derive the following three expressions

$$\mathcal{L}(\theta, \phi, x) = \int q_\phi(z|x) \log \frac{p_\theta(x, z)}{q_\phi(z|x)} dz \tag{2}$$

$$\mathcal{L}(\theta, \phi, x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{reconstruction performance averaged over } q_\phi(z|x)} - \underbrace{KL\left[q_\phi(z|x) \| p(z)\right]}_{\text{regularization: posterior } q_\phi(z|x) \text{ should be close to the prior}} \tag{3}$$

$$\mathcal{L}(\theta, \phi, x) = \mathbb{E}_{q_\phi(z|x)}[\underbrace{\log p_\theta(x|z)}_{\text{log-likelihood}} + \underbrace{\log p(z)}_{\text{prior}}] + \underbrace{H\left[q_\phi(z|x)\right]}_{\text{entropy term prevents collapse of } q_\phi \text{ to Dirac}} \tag{4}$$

Proof of (2), (3), (4)

$$\log p_\theta(x) = \log p_\theta(x) \int q_\phi(z|x)dz = \int q_\phi(z|x) \log p_\theta(x)\, dz\,, \text{ and } \quad KL\big[q_\phi(\cdot\,|x)\|p_\theta(\cdot\,|x)\big] \triangleq \int q_\phi(z|x) \log \frac{q_\phi(z|x)}{p_\theta(z|x)} dz$$

Hence from (1),

$$\mathcal{L}(\theta,\phi,x) = \int q_\phi(z|x)\left(\log p_\theta(x) - \log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right) dz = \int q_\phi(z|x)\left(\log \frac{p_\theta(z|x)p_\theta(x)}{q_\phi(z|x)}\right) dz = \int q_\phi(z|x) \log \frac{p_\theta(z,x)}{q_\phi(z|x)}\, dz$$

which proves (2). Then, since $p(z,x) = p(z)p(x|z)$ and $H\big[q_\phi\big] \triangleq -\int q_\phi(z|\mathrm{x}) \log q_\phi(z|x)dz$, we also have

$$\mathcal{L}(\theta,\phi,x) = \int q_\phi(z|x)\big(\log p_\theta(x|z) + \log p(z) - \log q_\phi(z|x)\big)dz = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z) + \log p(z)] + H\big[q_\phi\big]$$

which proves (4). Alternatively, $KL\big[q_\phi(z|x)\|p(z)\big] = \int q_\phi(z|\mathrm{x})\big(\log q_\phi(z|x) - \log p(z)\big)dz$, so from above we get:

$$\mathcal{L}(\theta,\phi,x) = \int q_\phi(z|x) \log p_\theta(x|z)\, dz - KL\big[q_\phi(z|x)\|p(z)\big], \quad \text{which proves (3).}$$

VAE loss function

- Hence for a data point x, the loss to **minimize** w.r.t. θ, ϕ is $-ELBO$:

$$-ELBO(\theta, \phi, x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}[-\log p_\theta(x|z)]}_{\text{reconstruction error}} + \underbrace{KL\left[q_\phi(z|x) \| p(z)\right]}_{\text{regularization}}$$

- For a dataset $X = \{x_i\}_{i=1\ldots N}$, recall that $\log p_\theta(X) = \sum_i \log p_\theta(x_i)$, thus we minimize

$$-ELBO(\theta, \phi, X) \triangleq \sum_{i=1}^{N} -ELBO(\theta, \phi, x_i)$$

- If we process a minibatch B, the minibatch estimate is $\frac{N}{|B|} \sum_{i \in B} -ELBO(\theta, \phi, x_i)$

VAE summary

1. We modelled the image synthesis process using a Bayesian generative model, with an observed variable and a latent variable

2. We wanted to maximize the marginal likelihood $p_\theta(X)$ to fit parameters, and to compute the posterior $p_\theta(z|x)$ to encode images, but this is intractable

3. We introduced a parametric approximation $q_\phi(z|x)$, called a variational posterior

4. We used $-ELBO(\theta, \phi, X)$ as a loss function to minimize w.r.t. θ, ϕ, which at the same time:
   - Ensures good approximations $q_\phi(z|x)$ i.e., good encoding
   - Ensures we learn a good data likelihood model $p_\theta(x|z)$ i.e., good generation

VAE loss in practice

- We want to use **backpropagation** to optimize the VAE
- The regularization term $KL\left[q_\phi(z|x)\|p(z)\right]$ can be computed in closed form when $q_\phi(z|x) \triangleq \mathcal{N}(z \mid \mu, \mathrm{diag}(\sigma)^2)$ where $(\mu, \sigma) = f_\phi(x)$, and $p(z) \triangleq \mathcal{N}(z|\, 0, \mathrm{I}_d)$:

$$KL\left[q_\phi(z|x)\|p(z)\right] := \frac{1}{2}\sum_{i=1}^{d}\left(\mu_j^2 + \sigma_j^2 - 1 - \log\left(\sigma_j^2\right)\right)$$

- What about the reconstruction error $\mathbb{E}_{q_\phi(z|x)}[-\log p_\theta(x|z)]$?
  - Inside the expectation is an MSE/SSD loss (Gaussian likelihood) or a cross-entropy loss (categorical likelihood), but we take it in expectation w.r.t. $q_\phi(z|x)$
- How to compute this term in an auto-differentiable manner?

VAE loss in practice

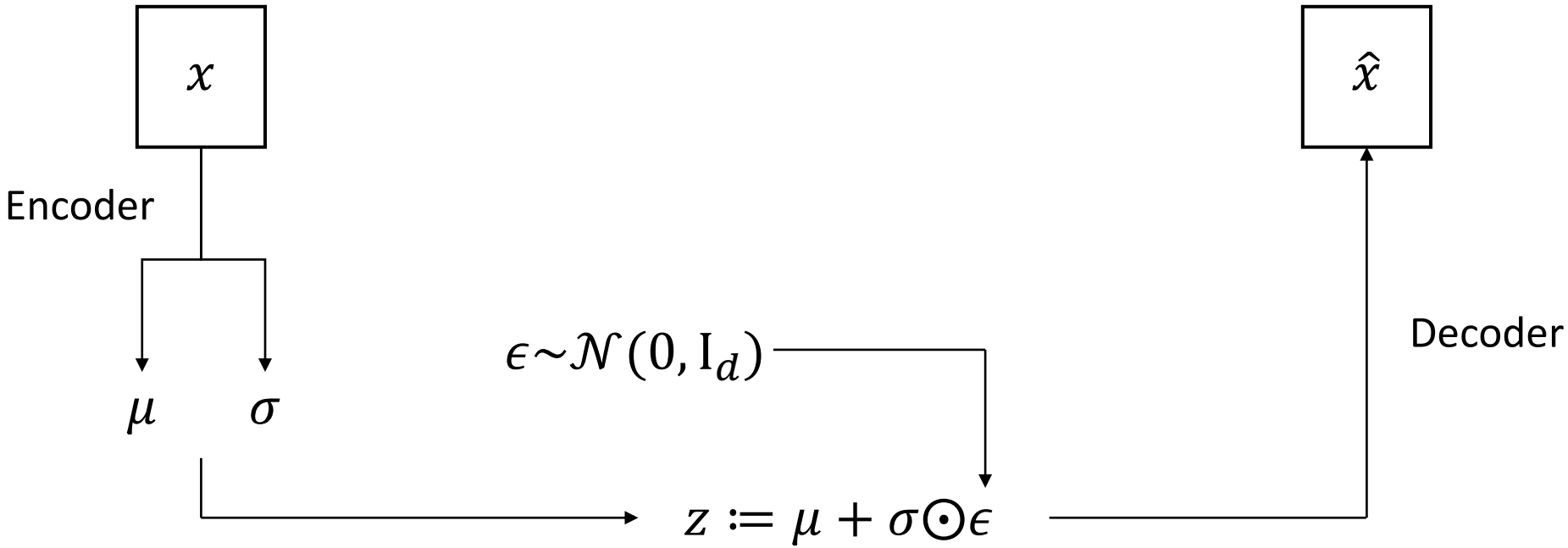
- We replace the expectation with a 1-sample stochastic estimate:

$$\mathbb{E}_{q_\phi(z|x)}[-\log p_\theta(x|z)] \approx -\log p_\theta(x|z), \quad \text{where } z \sim q_\phi(z|x)$$

- How do we sample z in a differentiable manner w.r.t. ϕ?
- Solution: **reparametrization trick**, express the sample via a differentiable functional relationship $h(\epsilon, \nu(\phi, x))$, where the randomness is introduced by a sample ϵ from a parameter-free random distribution
- **Example**: $q_\phi(z|x) \triangleq \mathcal{N}(z \mid \mu, \mathrm{diag}(\sigma)^2)$, with $(\mu, \sigma) = f_\phi(x)$ the output of the encoder.
  - Here, $\nu \triangleq (\mu, \sigma)$
  - We can sample $\epsilon \sim \mathcal{N}(0, \mathrm{I}_d)$ then compute $z := \mu + \sigma \odot \epsilon$ which has the correct distribution; as desired this is differentiable w.r.t. μ, σ

VAE in practice

- The variational auto-encoder is actually quite simple to implement
- Take the case of Gaussian $q_\phi(z|x)$



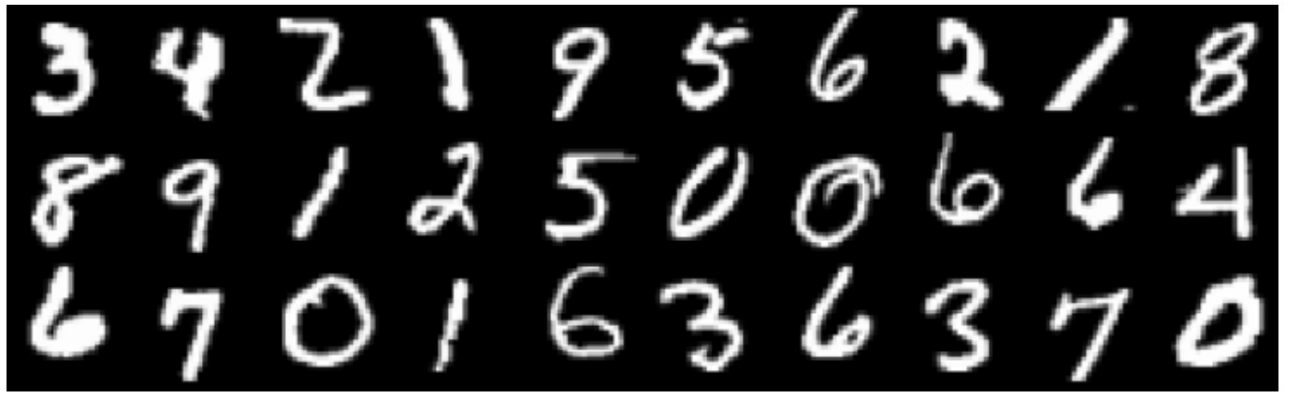
VAE in practice

- The final loss adapted to the MNIST dataset of binary images, noting $\hat{x} = g_\theta(z)$ the probability map output by the decoder, is

Reconstruction error

$$\sum_{p=1}^{mn} -x_p \log \hat{x}_p - (1 - x_p) \log(1 - \hat{x}_p) + \frac{1}{2} \sum_{j=1}^{d} \left(\mu_j^2 + \sigma_j^2 - 1 - \log\left(\sigma_j^2\right)\right)$$

Regularization



VAE in practice

- The final loss adapted to a dataset of natural images, noting $\hat{x} = g_\theta(z)$ the (mean) color image output by the decoder, is

Reconstruction error

$$\frac{1}{2\tau^2}\sum_{p=1}^{mn}\left\|x_p - \hat{x}_p\right\|^2 + \frac{1}{2}\sum_{j=1}^{d}\left(\mu_j^2 + \sigma_j^2 - 1 - \log\left(\sigma_j^2\right)\right)$$

Regularization



Variational auto-encoder results

- Some results of variational auto-encoders on MNIST data: random samples



(a) 2-D latent space (b) 5-D latent space (c) 10-D latent space (d) 20-D latent space

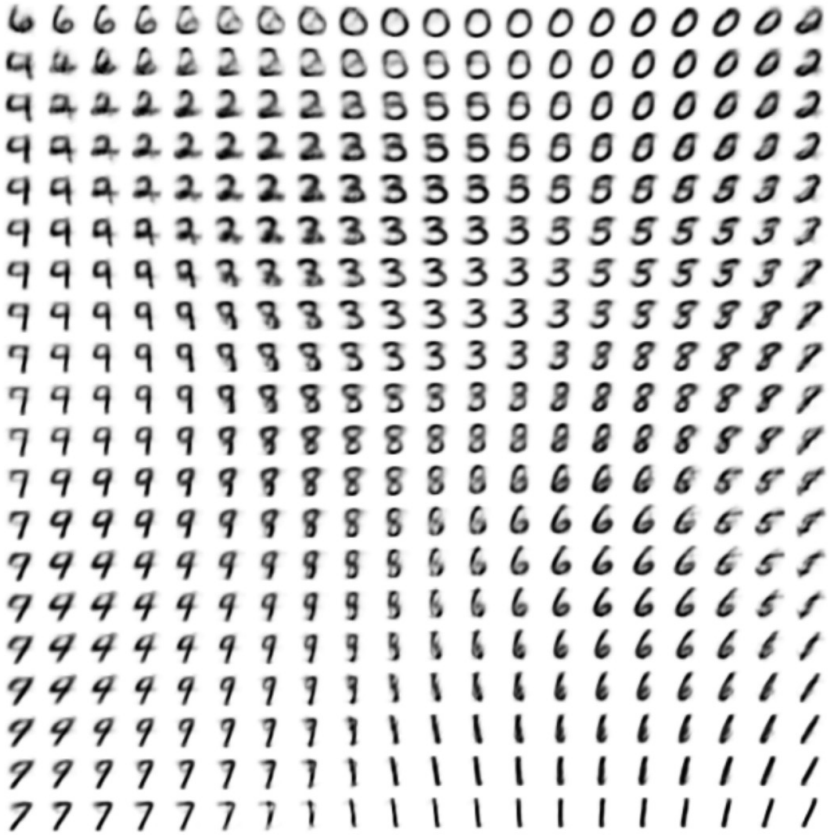
[Kingma, 2013] **Auto-encoding Variational Bayes**, Kingma, D. P. et Welling, M., *arXiv,* 2013

Variational auto-encoder results

- Some results of VAEs on MNIST and face data: uniform samples



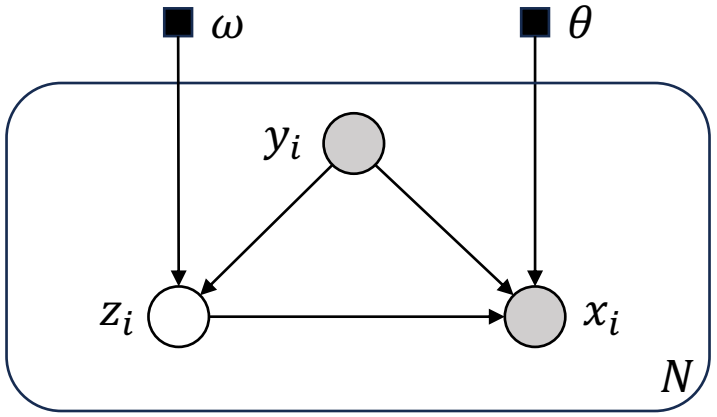
(a) Learned Frey Face manifold



(b) Learned MNIST manifold

[Kingma, 2013] **Auto-encoding Variational Bayes**, Kingma, D. P. et Welling, M., *arXiv,* 2013

Conditional VAEs (CVAEs)



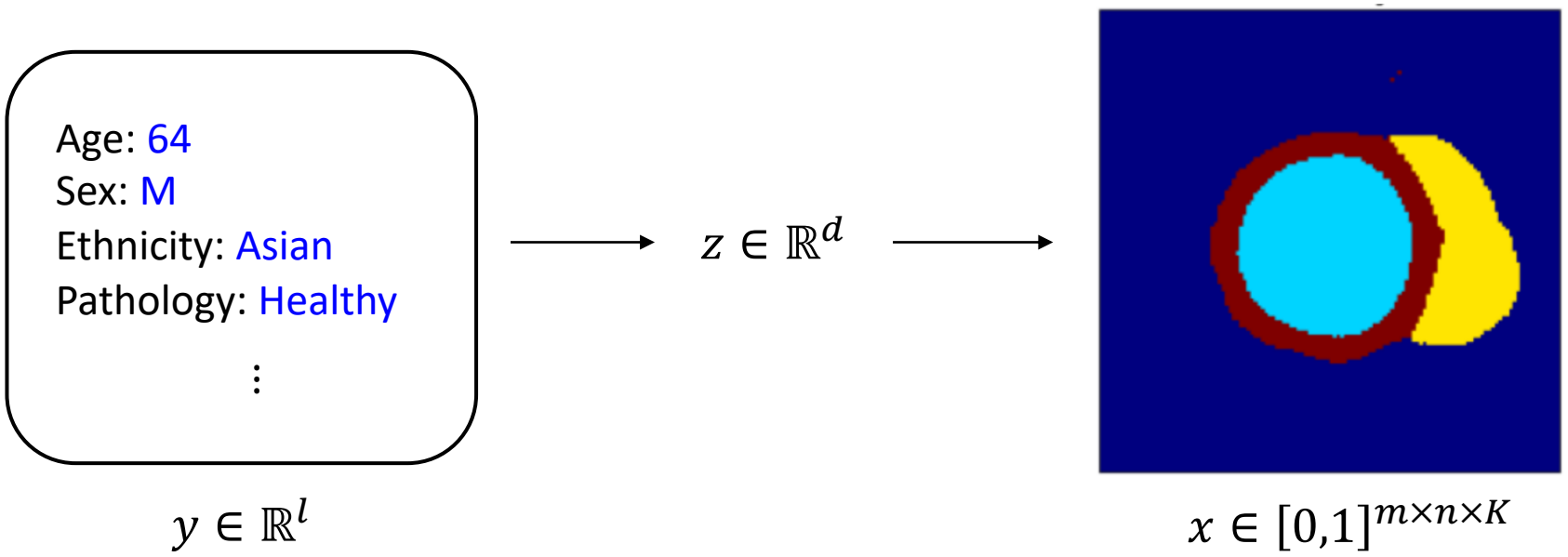
- In **CVAEs**, the prior $p_\omega(z|y)$ and the data likelihood $p_\theta(x|z,y)$ are conditioned on an additional observed variable y

- Both prior and likelihood can be modeled as neural networks to yield expressive models

[Sohn, 2015] **Learning structured output representation using deep conditional generative models**, Sohn, K. et al., *NIPS*, *2015*

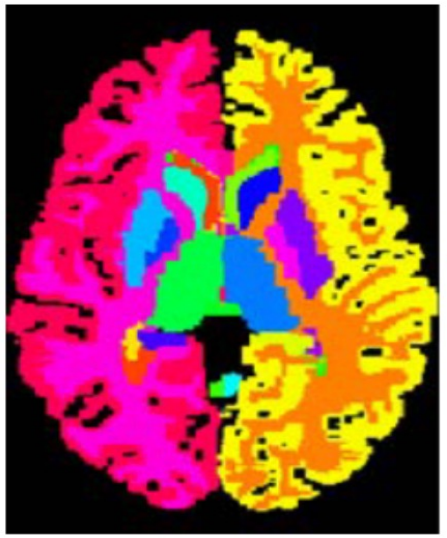
CVAEs promises in theory

- Generate a cardiac anatomy x given the latent representation z and known demographics & clinical data y



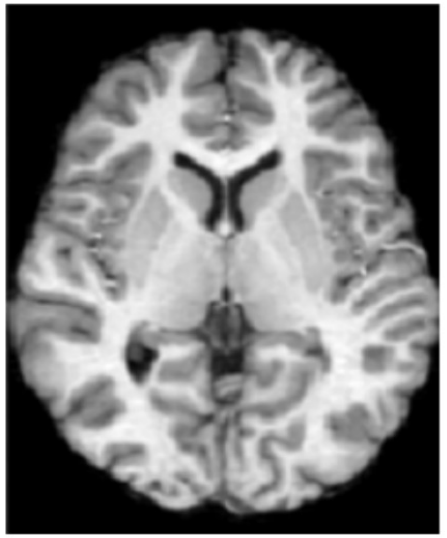
CVAEs promises in theory

- Generate a brain MRI x given the latent representation z and known brain anatomy y



$y \in \{0,1\}^{m \times n \times K}$

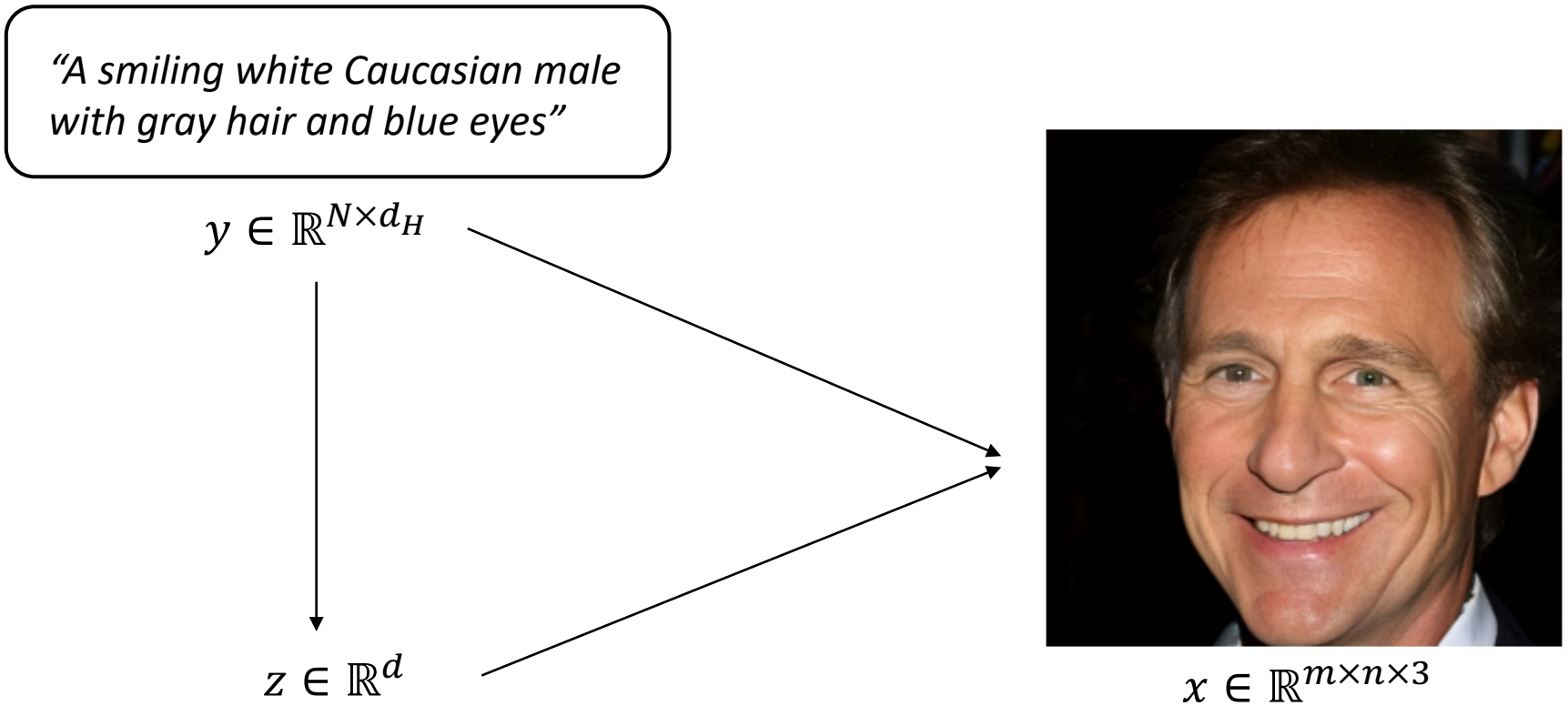
$z \in \mathbb{R}^d$



$x \in \mathbb{R}^{m \times n}$

CVAEs promises in theory

- Generate a face image x given the latent representation z and a caption y describing face attributes



CVAEs' training objective

- We maximize a lower bound to the marginal conditional likelihood $p_{\theta,\omega}(X|Y) \triangleq \int p_\theta(X|Z,Y)p_\omega(Z|Y)dZ$

- The ELBO for a data point (y, x) writes:

$$ELBO(\theta,\omega,\phi,x) = \underbrace{\mathbb{E}_{q_\phi(z|x,y)}[\log p_\theta(x|z,y)]}_{\text{reconstruction performance averaged over } q_\phi(z|x,y)} - \underbrace{KL\left[q_\phi(z|x,y)\|p_\omega(z|y)\right]}_{\text{regularization: posterior } q_\phi(z|x,y) \text{ should be close to the prior } p_\omega(z|y)}$$

VAEs: Problems and Solutions

Prior hole problem

- **Problem**: There is a discrepancy between the reconstructed images at training time and the generated images at test time

- At **test time**, we generate images in two steps

  1. Sample a latent code $z_* \sim p(z)$ according to the prior

  2. Sample an image $x_* \sim p_\theta(x|z_*)$

Prior hole problem

- **Problem**: There is a discrepancy between the reconstructed images at training time and the generated images at test time

- At **training time**, we generate reconstructed images in three steps

  1. Sample an index $n \sim q(n)$ uniformly in $\{1, \cdots, N\}$ to get a training image x_n
  2. Sample a latent code $z_n \sim q_\phi(z|n) \triangleq q_\phi(z|x_n)$
  3. Sample a reconstructed image $\hat{x}_n \sim p_\theta(x|z_n)$

Prior hole problem

- **Problem**: There is a discrepancy between the reconstructed images at training time and the generated images at test time

- At **training time**, we generate reconstructed images in three steps

  1. Sample an index $n \sim q(n)$ uniformly in $\{1, \cdots, N\}$ to get a training image x_n
  2. Sample a latent code $z_n \sim q_\phi(z|n) \triangleq q_\phi(z|x_n)$

  $\Big\}$ $z_n \sim q_\phi(z)$

  3. Sample a reconstructed image $\hat{x}_n \sim p_\theta(x|z_n)$

- Write $q_\phi(z) \triangleq \int q(z|n)q(n)dn$, it plays the role that $p(z) = \int p(z|x)p(x)dx$ plays at test time

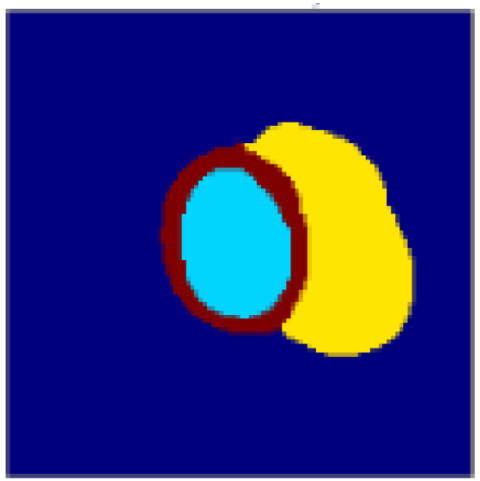
- We call $q_\phi(z)$ the *inferred prior*, or more commonly the **aggregate posterior**

Prior hole problem

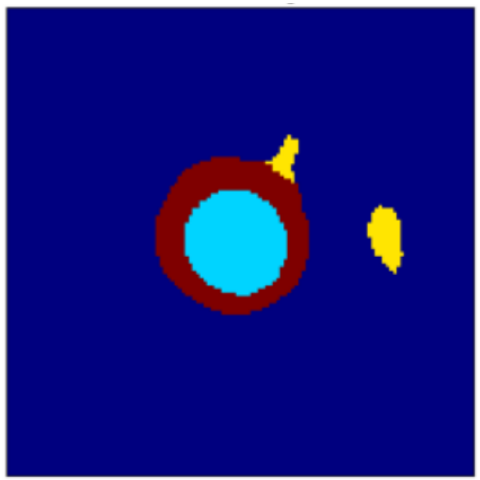
- **Problem**: There is a discrepancy between the reconstructed images at training time and the generated images at test time

- Indeed, if the variational approximation is imperfect: $q_\phi(z|x) \neq p_\theta(z|x) \rightarrow q_\phi(z)$ is far from $p(z)$

- As a result, generated images can be of reduced quality



Example reconstructed cardiac segmentation



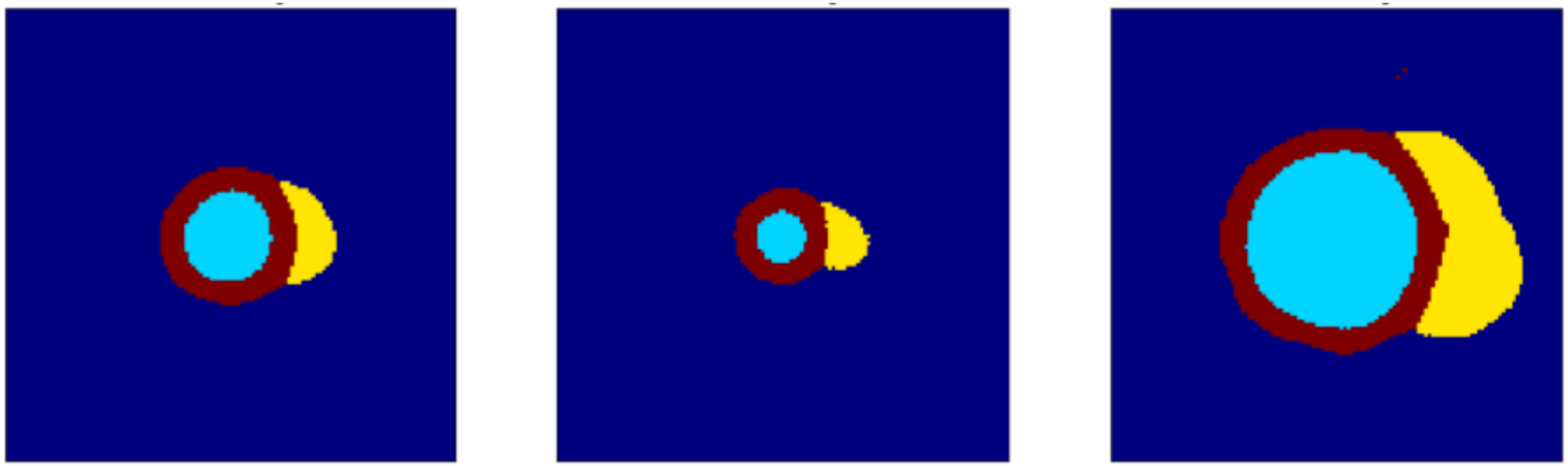
Example generated cardiac segmentation

β-VAE

- β-VAE adds a tunable hyperparameter β to weight differently the two terms in the ELBO

- This **changes the trade-off** between **reconstruction** error and **generation** quality
  - Generally, $\beta > 1$ to force the variational posterior closer to the prior

$$\mathcal{L} = \mathbb{E}_{q_\phi(z|x)}[-\log p_\theta(x|z)] + \beta \cdot KL\left[q_\phi(z|x) \| p(z)\right]$$



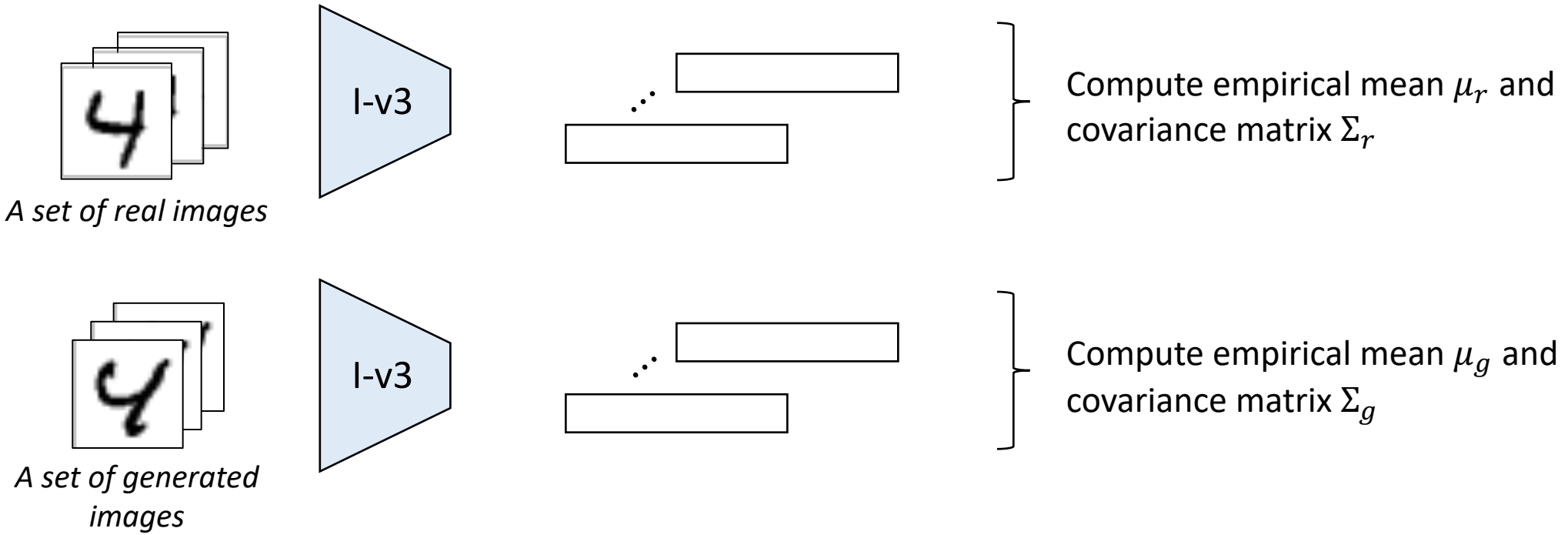
Example generated cardiac segmentations for a higher β value

[Higgins, 2017] **beta-vae: Learning basic visual concepts with a constrained variational framework**, Higgins, I. et al., *ICLR,* 2017

Quantifying the quality of generated images

- We can use the **Fréchet Inception Distance** (FID*)

- The FID **compares the statistics** of a set of real images to the statistics of a set of generated images
  - Specifically, it compares image **embeddings output by a trained CNN** (e.g., Inception-v3)



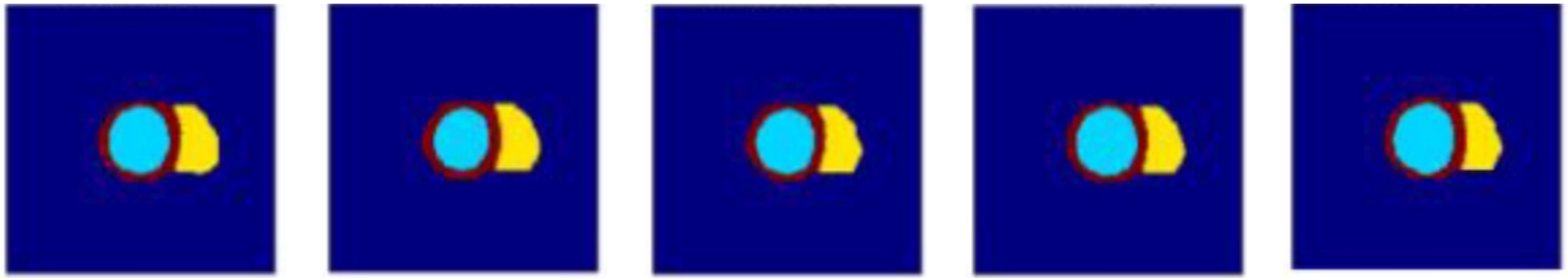
$$FID \triangleq \|\mu_r - \mu_g\|^2 + \mathrm{tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right)$$

*[Heusel, 2017] **GANs trained by a two time-scale update rule converge to a local nash equilibrium**, Heusel, M. et al., *NeuRIPS*, 2017

Posterior collapse

$$\mathcal{L} = \mathbb{E}_{q_\phi(z|x)}[-\log p_\theta(x|z)] + \beta \cdot KL\left[q_\phi(z|x)\|p(z)\right]$$

- When β is too large, **posterior collapse** can occur
  - The variational posterior $q_\phi(z|x)$ is forced very close to the prior $p(z)$ and cannot encode information about x → all reconstructed/generated images are virtually identical



Multiple reconstructed images for large β (similar results for generated images)

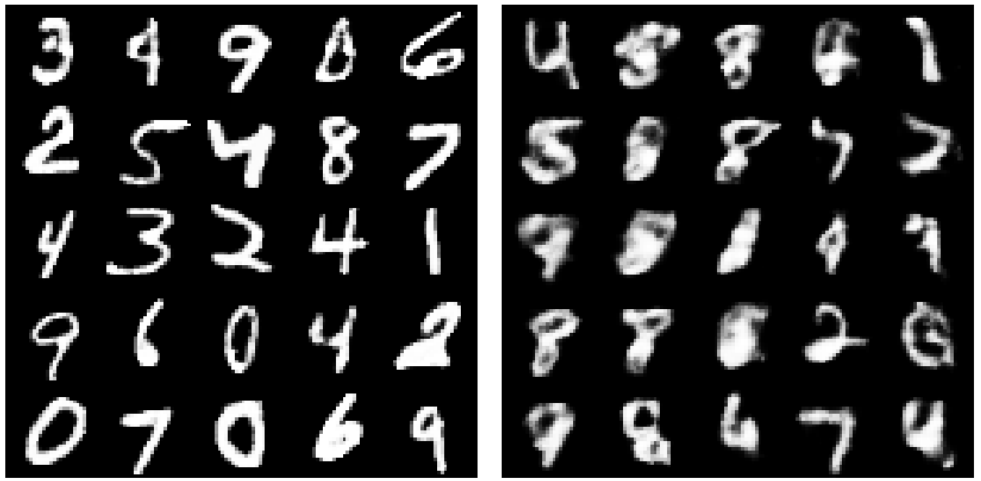
InfoVAE

- InfoVAE directly penalizes the **discrepancy** between the **aggregate posterior** and the **prior** by adding a term $KL[q_\phi(z)||p(z)]$ in the loss (and using a smaller β weight)

$$\mathcal{L} = \mathcal{L}_{\beta-\text{VAE}} + \gamma \cdot KL\left[q_\phi(z)\|p(z)\right]$$

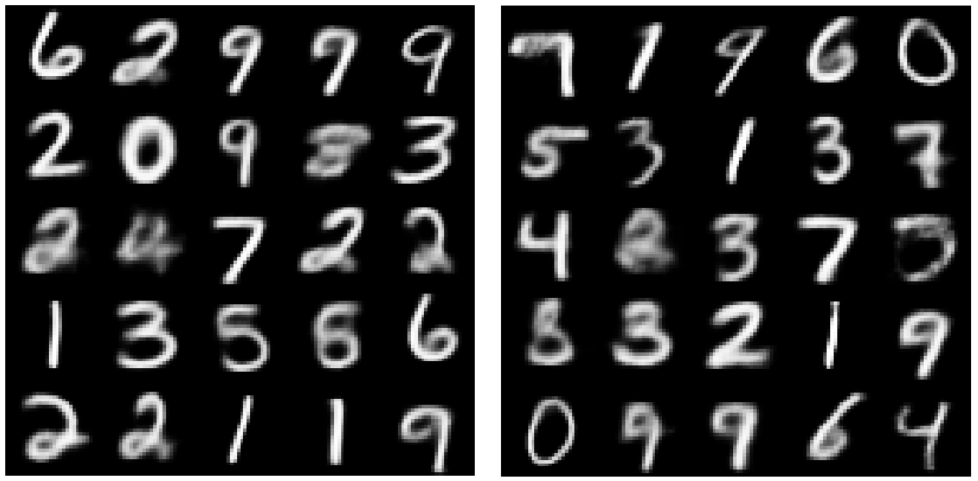
- The **log-odds output by a discriminator** D, jointly trained to discriminate samples from p from samples from q_ϕ, approximates this KL term (likelihood ratio trick)

β-VAE



reconstruction | generation

InfoVAE

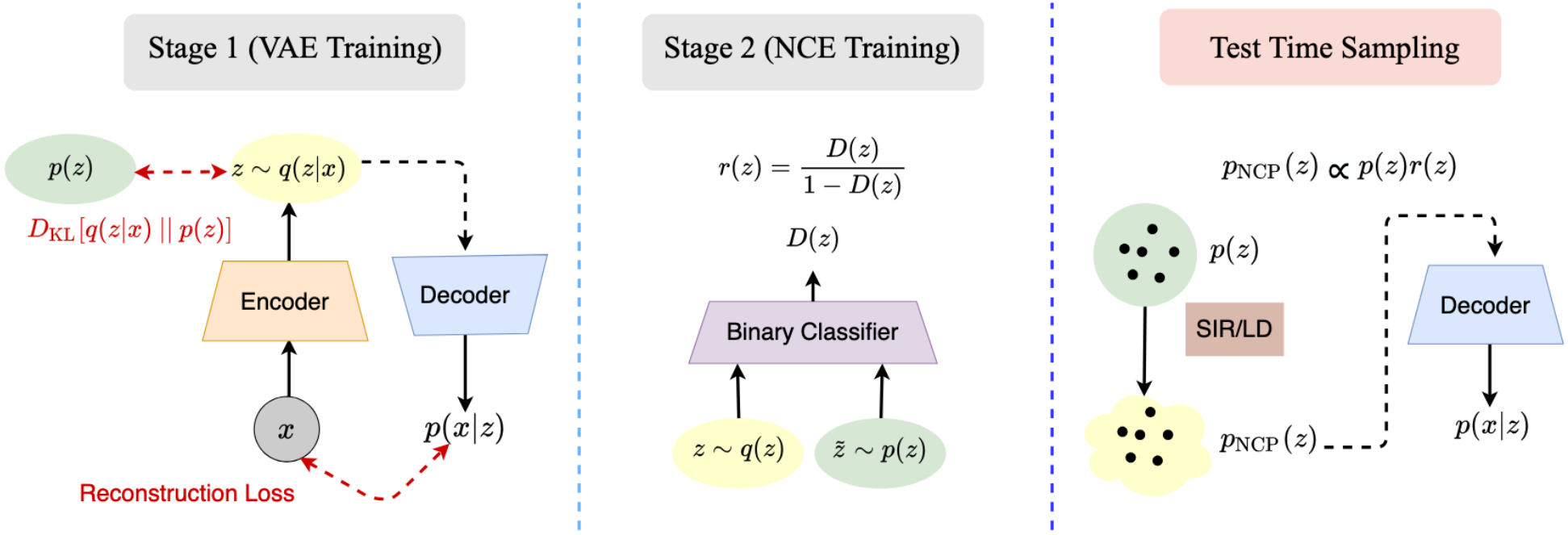


reconstruction | generation

[Zhao, 2019] **InfoVAE: Balancing Learning and Inference in Variational Autoencoders**, Zhao, S. et al., *AAAI,* 2019

Noise Contrastive Priors (NCP-VAE)

- Instead we can modify the prior $p(z)$ of a fixed, trained VAE model into $p_{\text{NCP}}(z) \propto r(z)p(z)$ so that $KL\left[q_\phi(z)\|p_{NCP}(z)\right] = 0$. This yields $r(z) = q_\phi(z)/p(z)$
- $r(z)$ can be estimated by the likelihood ratio trick (as a discriminator output, cf. previous slide)
- To sample from $p_{\text{NCP}}(z)$ at inference-time for generation, sampling-importance-resampling is used



[Aneja, 2021] **A Contrastive Learning Approach for Training Variational Autoencoder Priors**, Aneja, J. et al., *NeuRIPS,* 2021

Noise Contrastive Priors (NCP-VAE)

- Instead we can modify the prior $p(z)$ of a fixed, trained VAE model into $p_{\text{NCP}}(z) \propto r(z)p(z)$ so that $KL\left[q_\phi(z) \| p_{NCP}(z)\right] = 0$. This yields $r(z) = q_\phi(z)/p(z)$

- $r(z)$ can be estimated by the likelihood ratio trick (as a discriminator output, cf. previous slide)

- To sample from $p_{\text{NCP}}(z)$ at inference-time for generation, sampling-importance-resampling is used



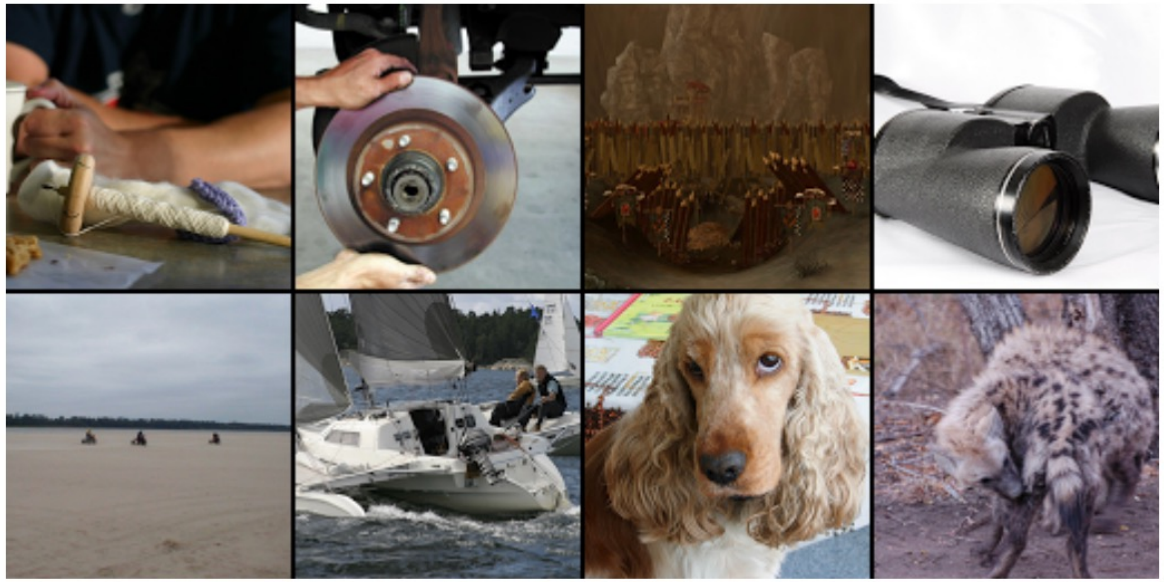


[Aneja, 2021] **A Contrastive Learning Approach for Training Variational Autoencoder Priors**, Aneja, J. et al., *NeuRIPS,* 2021

VQ-VAE

- Vector-Quantized VAEs avoid the difficulties of continuous latent spaces and propose to use **discrete latent variables** instead

- It is perhaps the most used VAE of the 2020's, mostly for **perceptual encoding**



Real images…



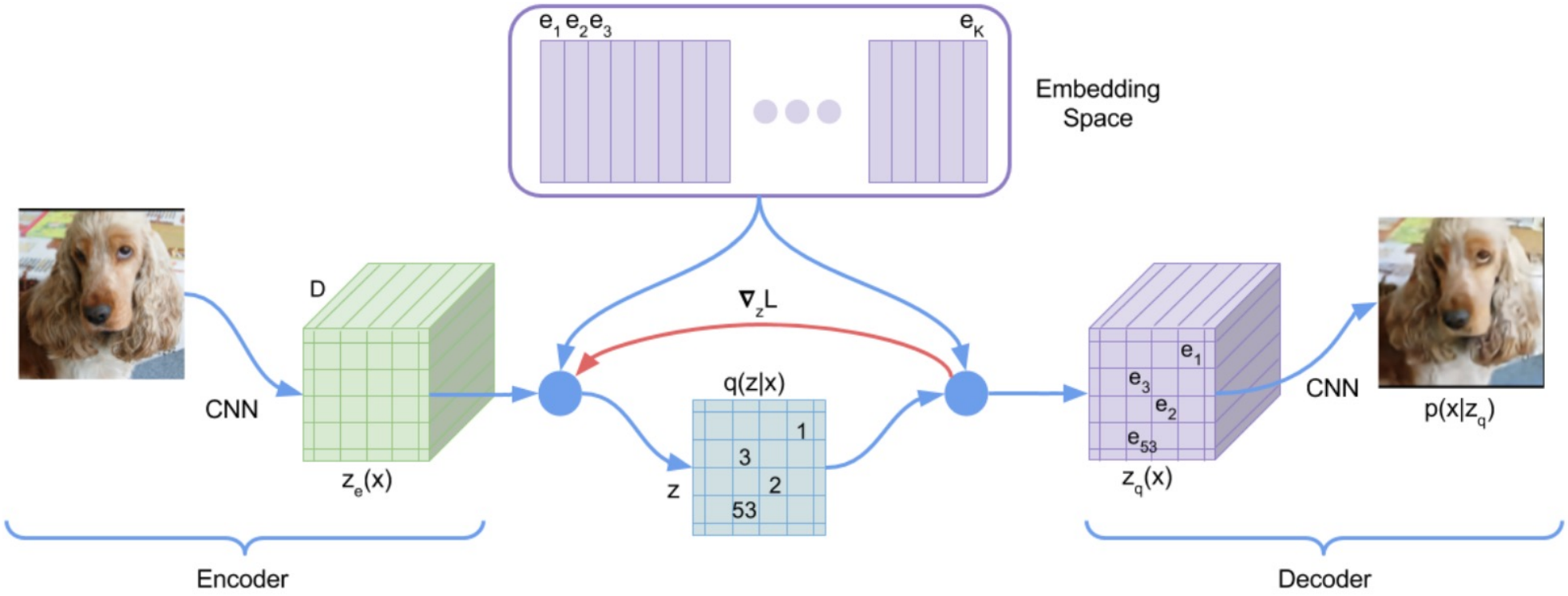
… and their reconstructions

[van den Oord, 2017] **Neural Discrete Representation Learning**, van den Oord, A. et al., *NeuRIPS,* 2017

VQ-VAE

- VQ-VAEs encode an $H \times W \times C$ image x to a $h \times w \times 1$ grid of integer variables $z(x)$, where integers map into a codebook with K codes (*e.g.*, $K \coloneqq 512$) for encoding-decoding (h is $H/4$ or $H/8$)

- A learnable prior $p_\psi(z)$ (PixelCNN, DALL-E, MaskGIT…) then needs to be separately fit on $\{z(x_i)\}_{i=1\ldots N}$ to turn a VQ-VAE into an actual generative model



[van den Oord, 2017] **Neural Discrete Representation Learning**, van den Oord, A. et al., *NeuRIPS,* 2017

Visual Token Prediction: DALL-E

- In DALL-E, the learnable prior $p_\psi(z)$ is an **autoregressive transformer** trained by likelihood maximization
  - In fact, DALL-E allows image generation conditioning on text and the prior is a joint autoregressive prior on visual (z) and text (y) tokens $p_\psi(y, z)$



(a) a tapir made of accordion. a tapir with the texture of an accordion.

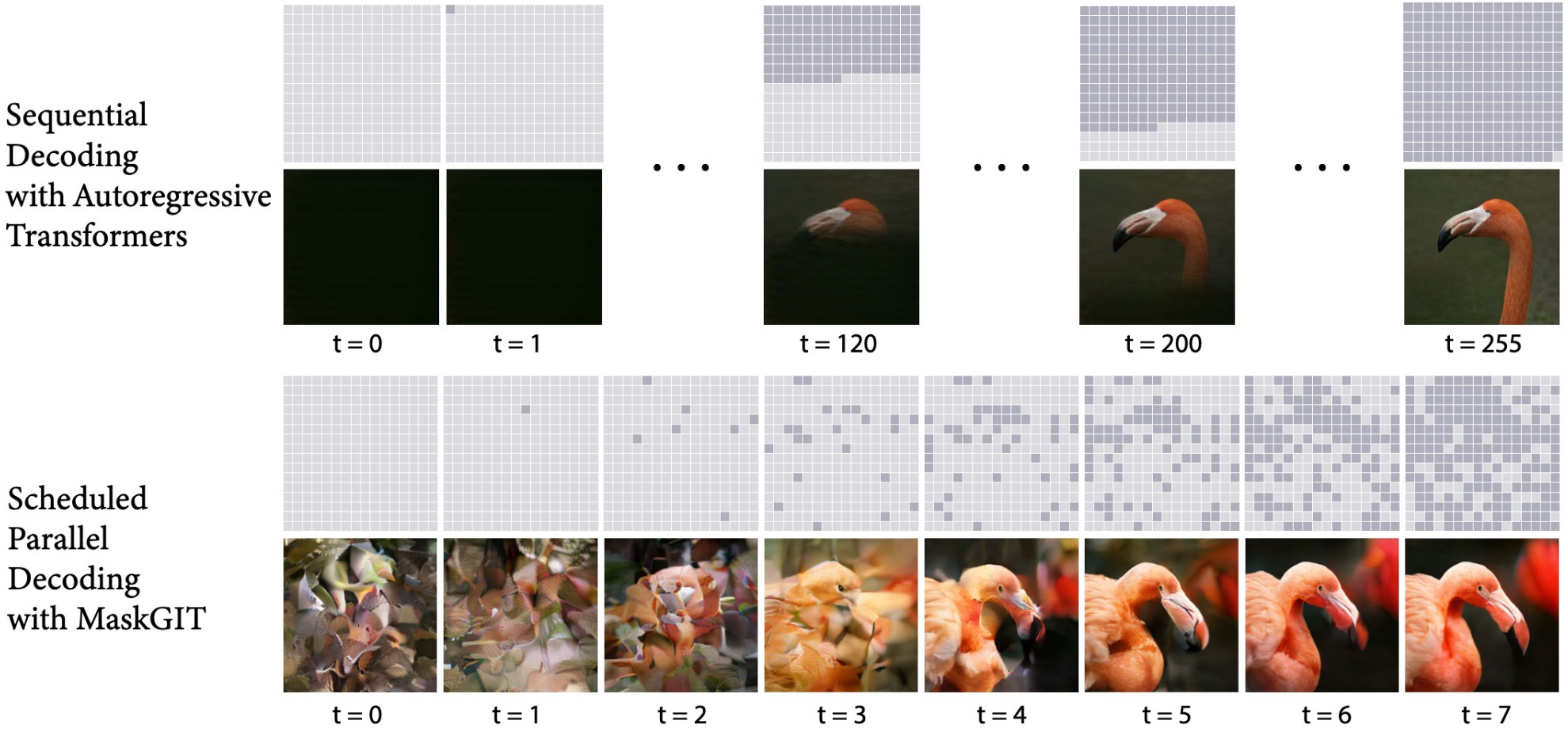
(b) an illustration of a baby hedgehog in a christmas sweater walking a dog

(c) a neon sign that reads "backprop". a neon sign that reads "backprop". backprop neon sign

[Ramesh, 2021] **Zero-Shot Text-to-Image Generation**, Ramesh, A. et al., *ICML,* 2021

Visual Token Prediction: MaskGIT

- In MaskGIT, the learnable prior $p_\psi(z)$ is a **bidirectional transformer** using a more efficient scheduled parallel decoding strategy for visual tokens, called **Masked Visual Token Modeling**



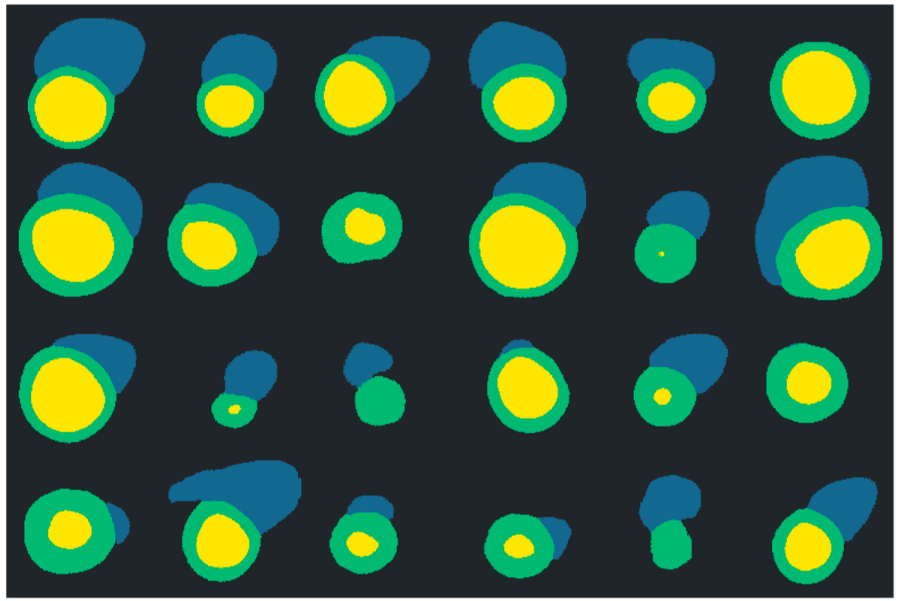
[Chang, 2022] **MaskGIT: Masked Generative Image Transformer**, Chang, H. et al., *CVPR,* 2022

Vanilla VAE problem: Blurry images

- If the model does not have enough capacity and images x, x' map to the same $q(z|x), q(z|x')$
  → the **pixel-level reconstruction loss** encourages the decoder to map to the mean of x and x'



CelebA β-VAE generations

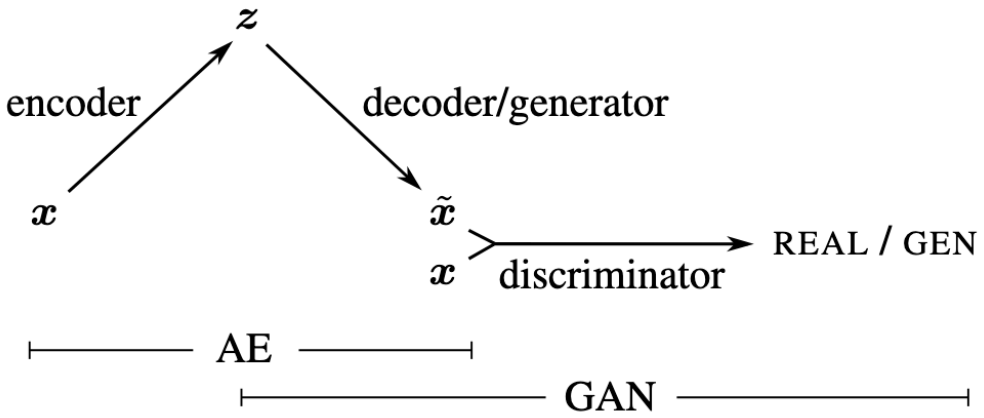


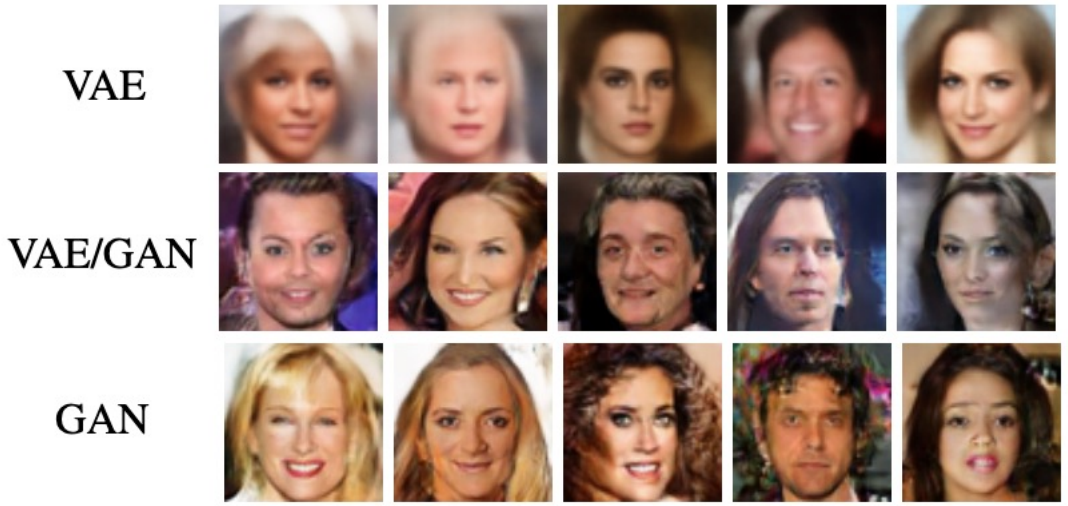
Cardiac shapes β-VAE generation

[Jiang, 2024] **Cardiac shape analysis with Nouveau VAE**, Jiang, F, *MSc thesis, ICL, 2024*

Adversarial losses

- Idea: use an **adversarial loss à la GAN** instead, within an autoencoder framework
  - VAE/GAN [Larsen, 2016] was the first to propose this, with a GAN loss and a perceptual reconstruction loss in the latent space of the discriminator



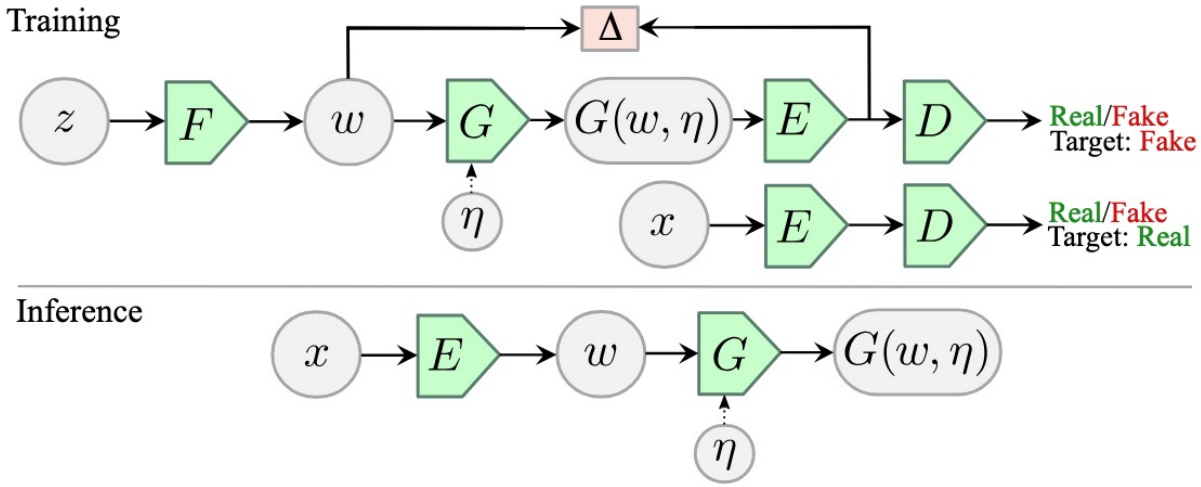


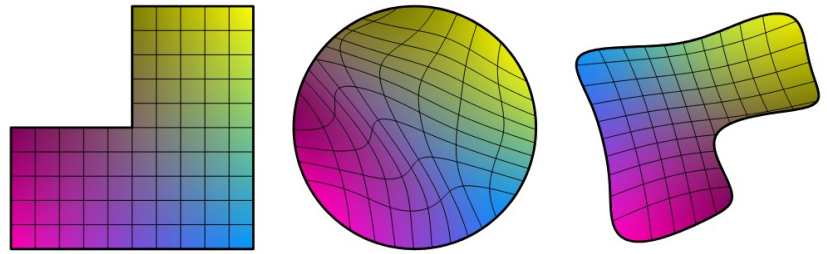
- Later methods like (Soft)IntroVAE [Daniel, 2021] or DC-VAE [Parmar, 2021] propose various adversarial or contradistinctive losses, plus couple the discriminator and the encoder for efficiency

[Larsen, 2016] **Autoencoding beyond pixels using a learned similarity metric**, Larsen, A.B.L. et al., *ICML,* 2016
[Daniel, 2021] **Soft-IntroVAE: Analyzing and Improving the Introspective Variational Autoencoder**, Daniel, T. et al., *CVPR,* 2021
[Parmar, 2021] **Dual Contradistinctive Generative Autoencoder**, Parmar, G. et al., *CVPR,* 2021

ALAE / StyleALAE

- **ALAE** [Pidhorskyi, 2020] **brings to autoencoders** much of what made **StyleGAN** state-of-the-art
- The latent space $w \in \mathcal{W}$ (for attribute manipulation…) is distinct from the noise $z \in \mathcal{Z}$ that is sampled to generate images; w is used for autoencoding





(a) Distribution of features in training set

(b) Mapping from $\mathcal{Z}$ to features

(c) Mapping from $\mathcal{W}$ to features

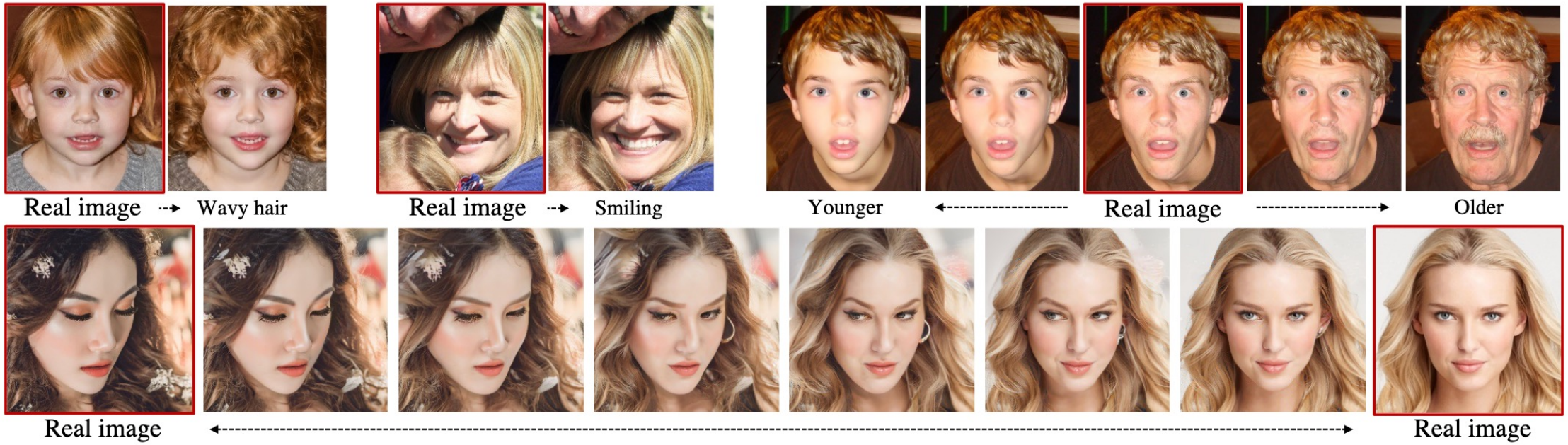
Illustration with 2 latent variables

- ALAE combines an adversarial loss, and a reconstruction loss in latent space

[Pidhorskyi, 2020] **Adversarial Latent Autoencoders**, Pidhorskyi, S. et al., *CVPR,* 2020
[Karras, 2019] **A Style-Based Generator Architecture for Generative Adversarial Networks**, Karras, T. et al., *CVPR,* 2019

Diff-AE

- Diff-AE [Preechakul, 2022], a diffusion autoencoder, also uses a $\mathcal{W}$ space for attribute manipulation
- When generating images, the mapping from noise to the $\mathcal{W}$ space is done by another diffusion model!

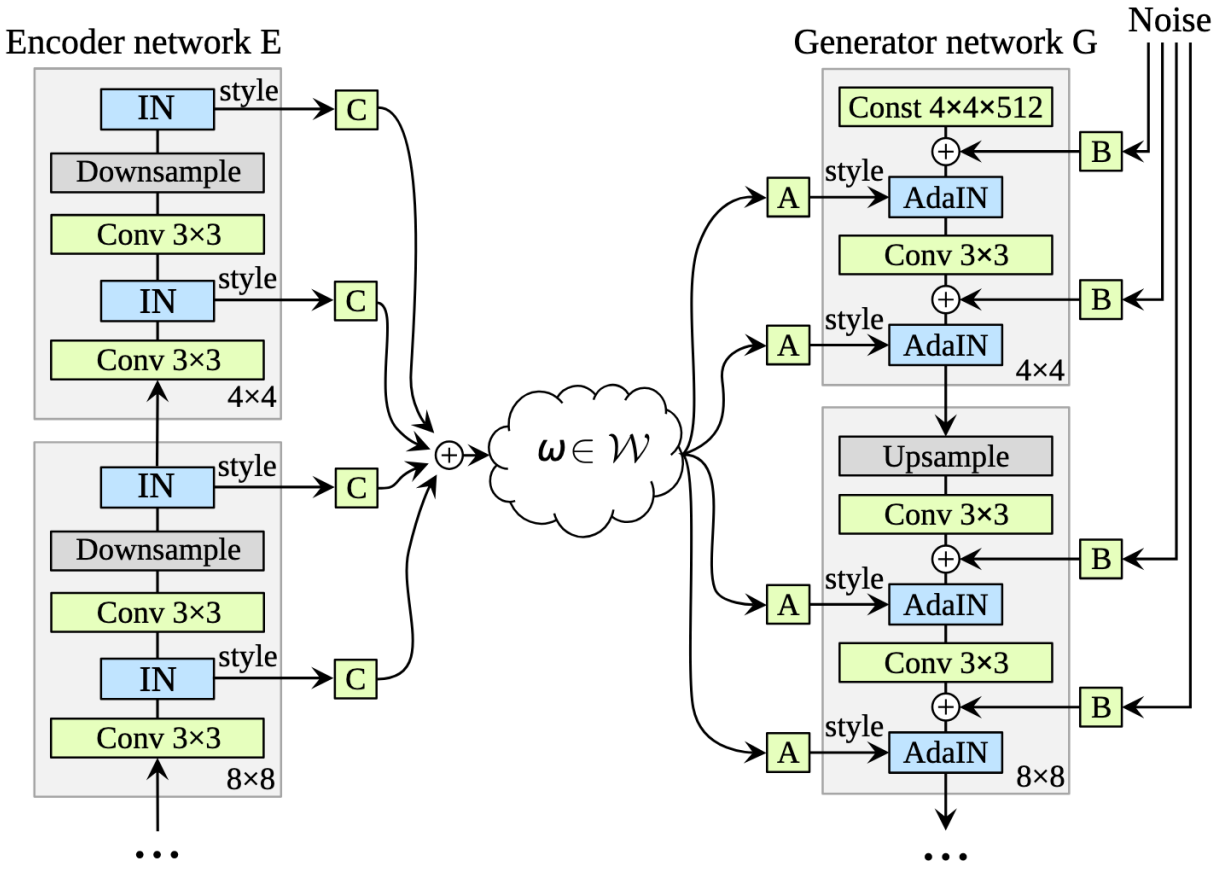


More results here!

[Preechakul, 2022] **Diffusion Autoencoders: Toward a Meaningful and Decodable Representation**, Preechakul, K. et al., *CVPR*, *2022*

ALAE / StyleALAE: architecture

- StyleALAE's decoder mimics StyleGAN; a matching encoder is designed
  - The autoencoding process is done by **Adaptive** (Instance) **Normalization layers**



AdaIN:

$$y := \sigma(w) \cdot \frac{x - \mu}{\sigma} + \mu(w)$$

Normalization depends on latent code (implemented as MLP)

[Pidhorskyi, 2020] **Adversarial Latent Autoencoders**, Pidhorskyi, S. et al., *CVPR,* 2020

ALAE / StyleALAE: reconstruction

- StyleALAE's and other adversarial autoencoders' **reconstruction capabilities are not ideal**; this is significantly improved upon by hierarchical VAEs (see next) or diffusion autoencoders

StyleALAE inputs/reconstructions



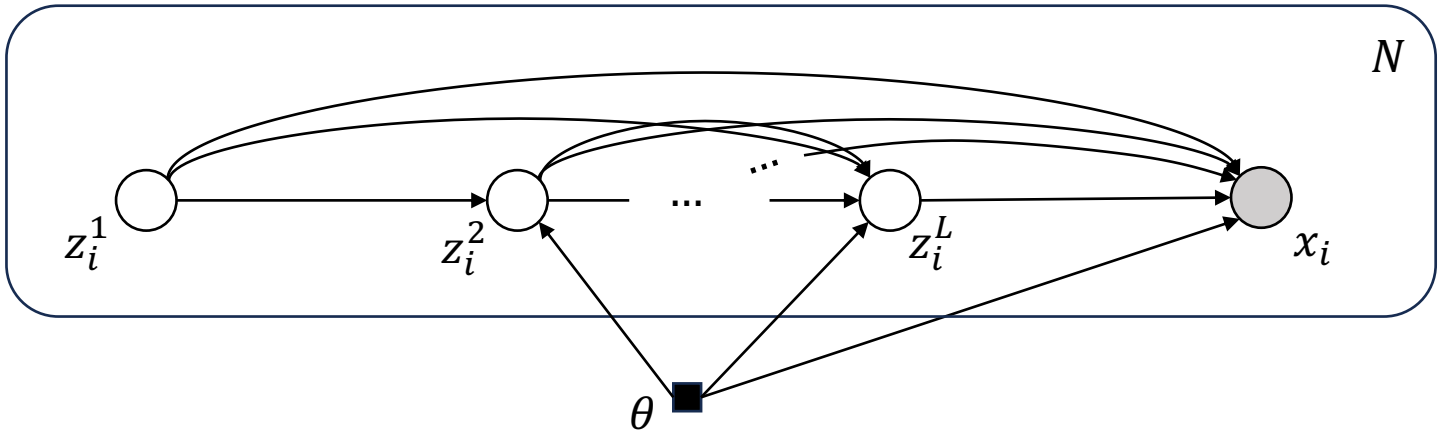
Diff-AE inputs/reconstructions



Inputs Stochastic reconstructions

[Pidhorskyi, 2020] **Adversarial Latent Autoencoders**, Pidhorskyi, S. et al., *CVPR,* 2020
[Preechakul, 2022] **Diffusion Autoencoders: Toward a Meaningful and Decodable Representation**, Preechakul, K. et al., *CVPR*, *2022*

Hierarchical VAE



- Hierarchical VAEs introduce a hierarchy (L layers) of latent variables $z^1, \cdots, z^L$, with increasing numbers of variables at each layer; each layer depends on every layer before in the hierarchy

$$p_\theta(x, z^1, \cdots, z^L) \triangleq p(z^1)p_\theta(z^2|z^1) \cdots p_\theta(z^L|z^{<L})p_\theta(x|z^L, \ldots, z^1)$$

- NVAE [Vahdat, 2020] scales up to $L \coloneqq 36$ layers, with $8\times8\times20$ variables in the first layer and $128\times128\times20$ in the last

[Vahdat, 2020] **NVAE: A deep hierarchical variational autoencoder**, Vahdat, A. et Kautz, J., *NeuRIPS*, *2020*

Hierarchical VAE

We can visualize the impact of a specific layer l by fixing everything up to layer $l-1$ and sampling from l (below, layers are grouped by scale, 1st scale $= 8\times8$, 5th scale $= 128\times128$)



[Vahdat, 2020] **NVAE: A deep hierarchical variational autoencoder**, Vahdat, A. et Kautz, J., *NeuRIPS*, *2020*

Hierarchical VAE

- The variational posterior is chosen to **mirror the hierarchy** in the generative model:

$$q_\phi(z^1, \dots, z^L|x) \triangleq q_\phi(z^1|x) q_\phi(z^2|z^1, x) \cdots q_\phi(z^L|z^{<L}, x)$$

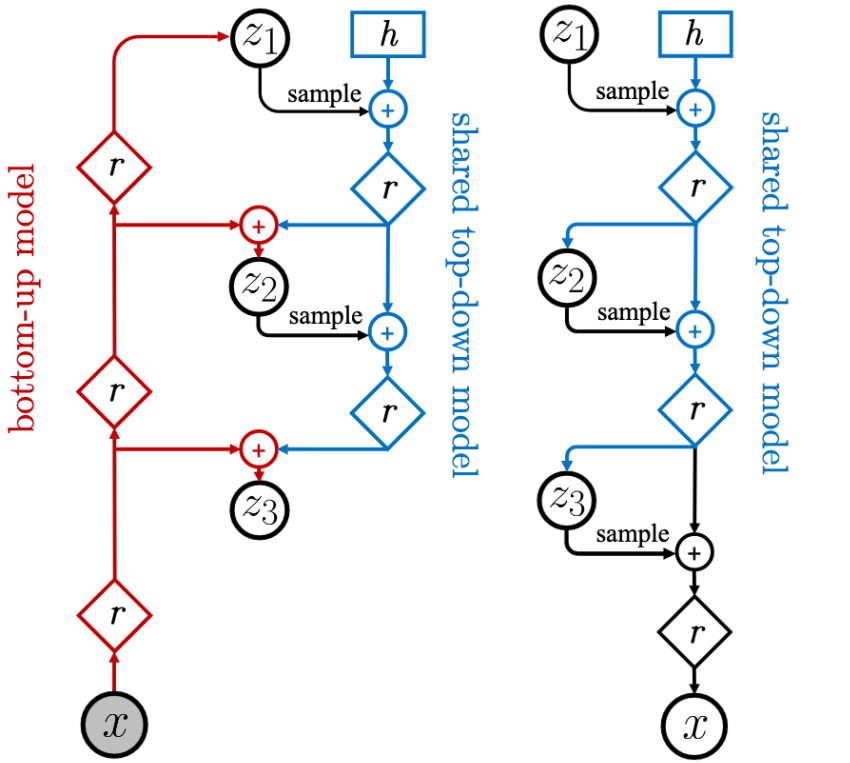
- $KL\left[q_\phi(z^1, \dots, z^L|x) || p_\theta(z^1, \dots, z^L)\right]$ in the ELBO decomposes over each factor:

$$\begin{aligned} & ELBO(\theta, \phi, x) \\ & = \mathbb{E}_{q_\phi(z|x,y)}[\log p_\theta(x|z^{1:L})] - KL\left[q_\phi(z^1|x) \| p(z^1)\right] \\ & - \sum_{l=2}^{L} \mathbb{E}_{q_\phi(z^{<l}|x)} \left[KL\left[q_\phi\left(z^l|z^{<l}, x\right) \| p_\theta(z^l|z^{<l})\right]\right] \end{aligned}$$

[Vahdat, 2020] **NVAE: A deep hierarchical variational autoencoder**, Vahdat, A. et Kautz, J., *NeuRIPS, 2020*

Hierarchical VAE: architecture

For $q_\phi(z|x)$, we require a bottom-up deterministic network to extract representations from x, and an additional top-down network to infer latent variables z^l in each layer starting from z^1



(a) Bidirectional Encoder (b) Generative Model

Figure 2: The neural networks implementing an encoder $q(\boldsymbol{z}|\boldsymbol{x})$ and generative model $p(\boldsymbol{x}, \boldsymbol{z})$ for a 3-group hierarchical VAE. ◇ denotes residual neural networks, ⊕ denotes feature combination (e.g., concatenation), and $\boxed{h}$ is a trainable parameter.

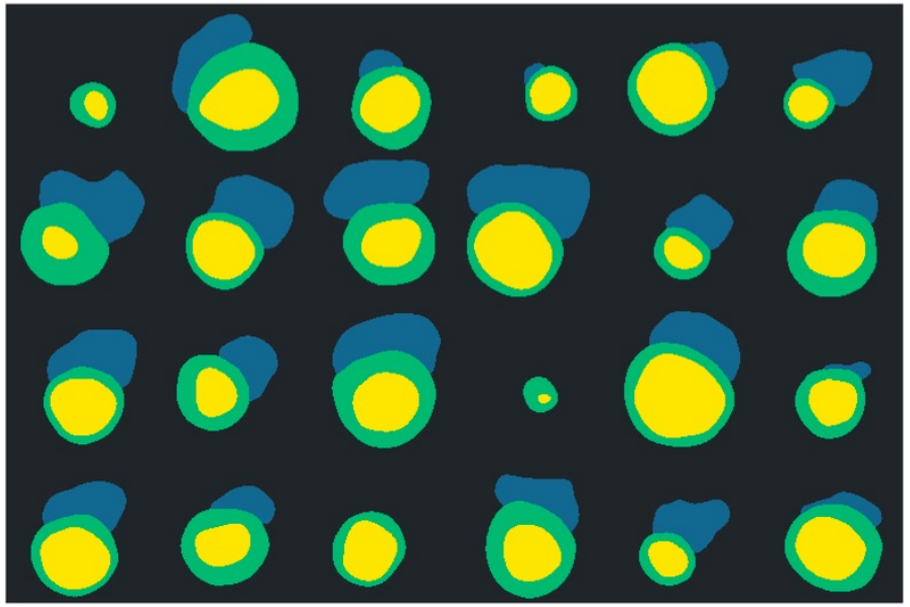
[Vahdat, 2020] **NVAE: A deep hierarchical variational autoencoder**, Vahdat, A. et Kautz, J., *NeuRIPS, 2020*

Hierarchical VAE: examples of generated images

CelebA-HQ generations



Cardiac shapes generation



- NVAE is also among the SOTA in terms of reconstruction capabilities

[Vahdat, 2020] **NVAE: A deep hierarchical variational autoencoder**, Vahdat, A. et Kautz, J., *NeuRIPS*, *2020*
[Jiang, 2024] **Cardiac shape analysis with Nouveau VAE**, Jiang, F., *MSc thesis, ICL*, *2024*

Conditioning in Generative Models

How to condition on latent codes / auxiliary inputs?

1. For models that first map the latent code to low-dimensional feature maps (vanilla VAE) or to spatial tokens (TransGAN), simple MLPs are used

2. For introducing a condition at several stages in the decoding process (time or class for diffusion models): (a) map the condition to an embedding, (b) broadcast and add/concatenate to the feature maps

3. Alternatively use Adaptive Instance/Group/Layer Normalization layers, whose μ's and σ's depend on the condition via a small MLP: this is used in StyleALAE, ViTGAN, SODA diffusion, Diff-AE, etc.

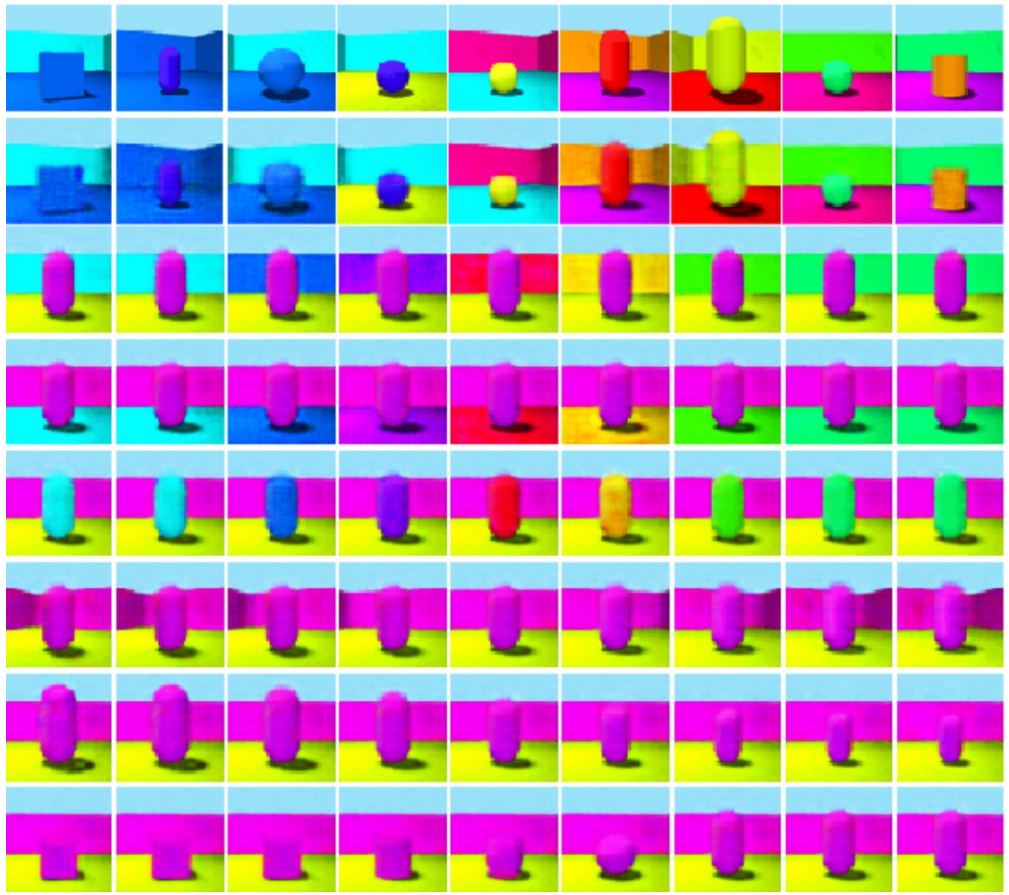
4. For models using transformer blocks, additional tokens (for the conditions) can be concatenated to the sequence of spatial tokens: thus all tokens can attend to each other in SA layers; this is done in DALL-E and many LM-based generative models like 4M, etc.

5. For conditioning on text or other images, encoded as token sequences, cross-attention layers can be added to the model: this is used in StableDiffusion and many more to condition on text

Disentanglement

Disentanglement

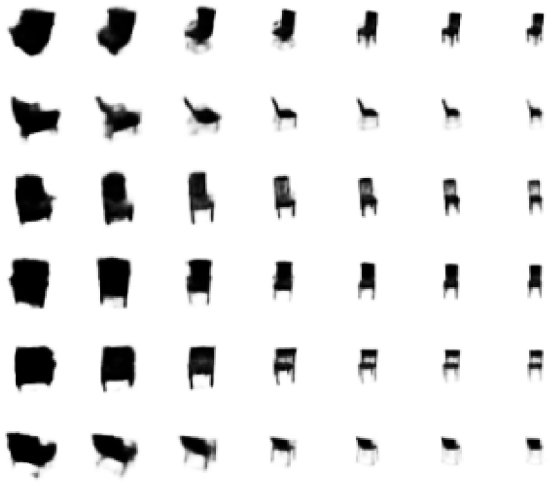
- In a disentangled representation, a single latent variable z_i is sensitive to changes in a **single** human-interpretable, **semantic generative factor**; while being invariant to changes in other factors



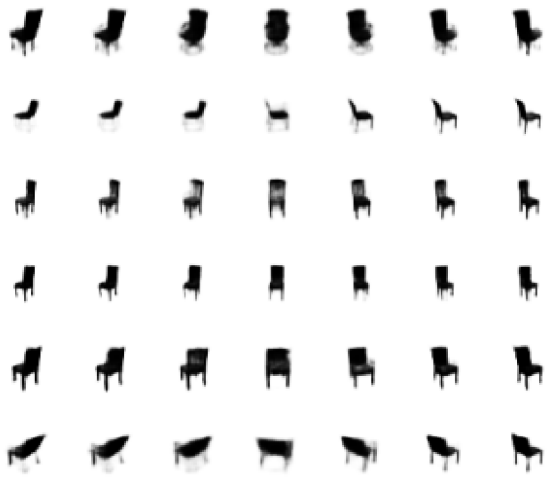
[Kim, 2018] **Disentangling by factorizing**, Kim, H. et Mnih, A., *ICML, 2018*

Examples of semantic factors of variation

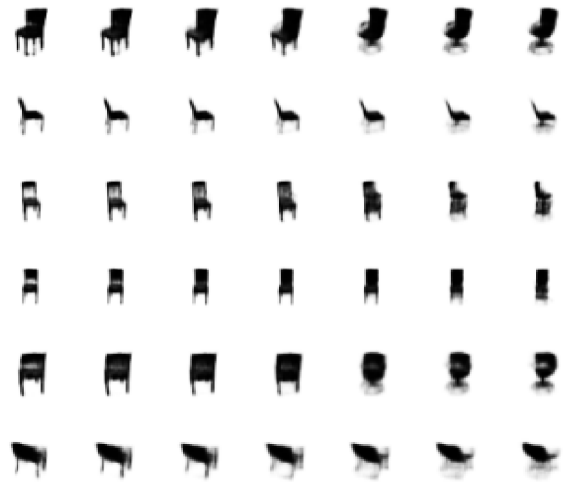
- For a dataset of objects: object type, position, scale, lighting or color;
- For faces: face width, hair color, hair style, sex, beard, moustache, glasses, lighting, emotion, azimuth…
- Note that some factors may be correlated e.g., sex and beard



Chair size



Azimuth



Leg style

[Higgins, 2017] **beta-vae: Learning basic visual concepts with a constrained variational framework**, Higgins, I. et al., *ICLR,* 2017

Disentanglement

- When the representation is disentangled, it is easier to modify a single specific factor of variation





Videos available here!

[Preechakul, 2022] **Diffusion Autoencoders: Toward a Meaningful and Decodable Representation**, Preechakul, K. et al., *CVPR*, *2022*

Disentanglement

Disentangled representations have several potential advantages:

i. **Generalization:** they can generate novel combinations of the underlying generative factors; e.g., train on ‘big green apple’ and ‘small red apple’, and generate a ‘big red apple’

ii. **Invariance:** it is easy to derive a representation invariant to nuisance factors by simply marginalizing over the corresponding latent variables

iii. **Transferability:** the fact that the key generative factors of the observed distributions are seggregated along different latents may enable easier transfer to new distributions where specific factors are modified

iv. **Interpretability:** human observers may be able to assign meaning to the dimensions

v. **Conditioning and intervention:** they allow for interpretable conditioning and/or intervention over a subset of the latents to observe its effects on the generated data

[Kumar, 2018] **Variational inference of disentangled latent concepts from unlabeled observations**, Kumar, A. et al., *ICLR,* 2018

Learning disentangled representations

There are several types of methods:

i. **Supervised disentanglement:** the semantic generative factors are (partly or entirely) known and we use this knowledge explicitly to guide the representation learning

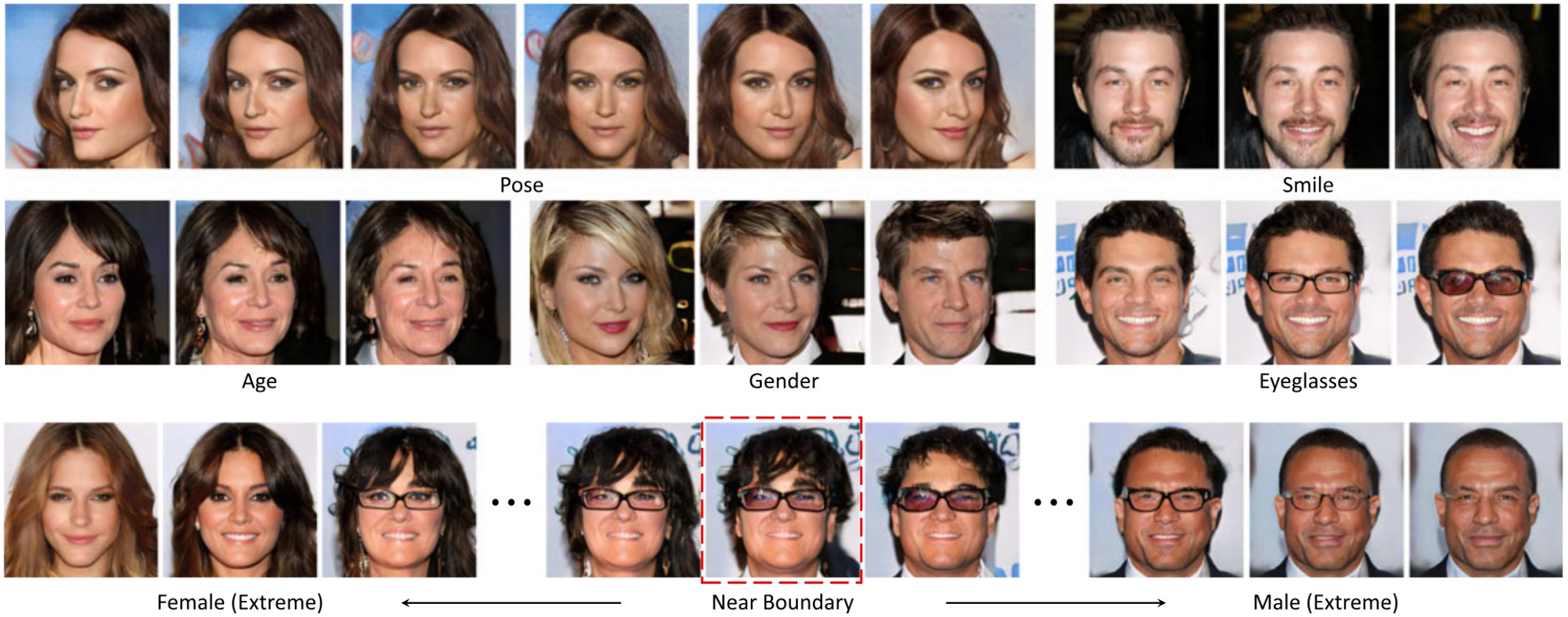
ii. **Weakly-supervised disentanglement:** we only have observed data, but some additional information is available (label, grouping of observations, number of varying factors, etc.) that may help disentanglement

iii. **Unsupervised disentanglement:** we aim to learn a disentangled representation purely from the observed data, without any form of supervision

We will only discuss the third category, which is well suited to VAEs

A word on supervised disentanglement

- The most effective methods are supervised: they learn a separating hyperplane in latent $\mathcal{W}$ space and move the samples in this direction (e.g., InterFaceGAN)



[Shen, 2022] **InterFaceGAN: Interpreting the Disentangled Face Representation Learned by GANs**, Shen, Y. et al., *IEEE PAMI*, *2022*

β-VAE

- β-VAE [Higgins, 2017] was originally introduced for disentangled representation learning

$$\mathcal{L} = \mathbb{E}_{q_\phi(z|x)}[-\log p_\theta(x|z)] + \beta \cdot KL\left[q_\phi(z|x)\|p(z)\right]$$

- It is argued that controlling the capacity of latent channels i.e., the amount of information that can be encoded about x, forces an efficient encoding that favors disentangled representations

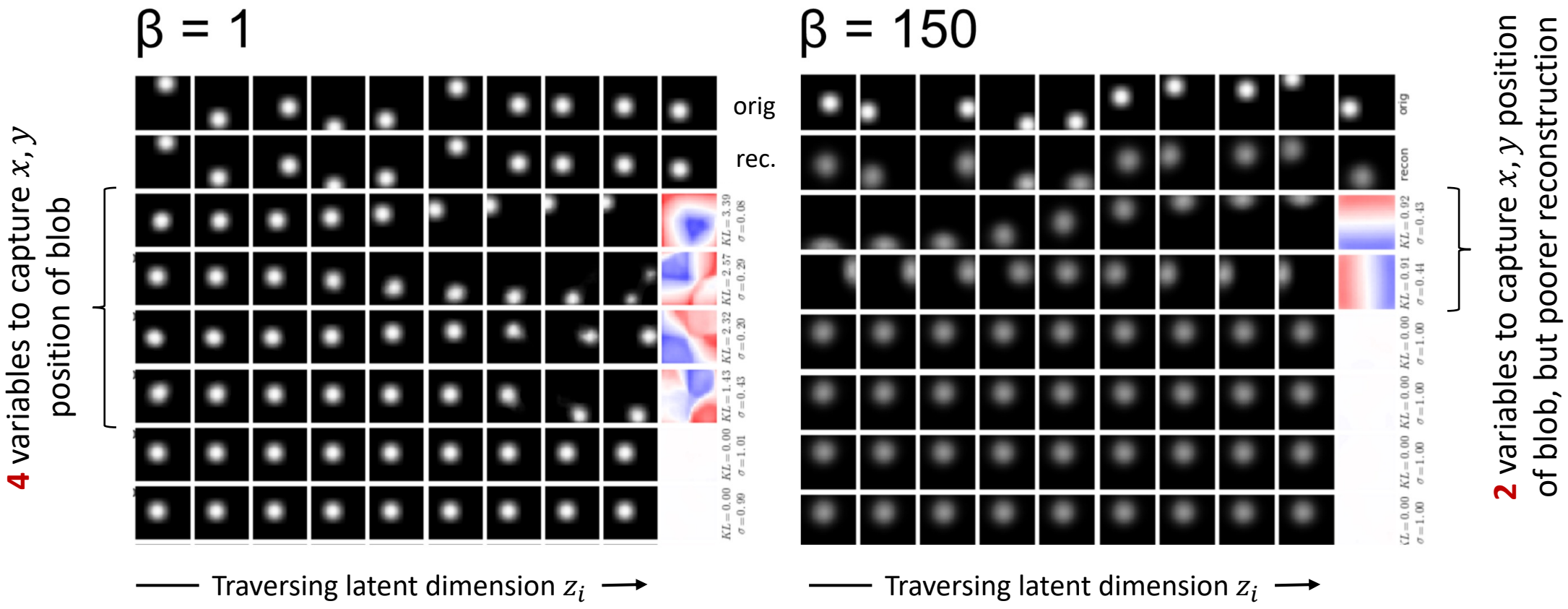


β-VAE separates azimuth fairly well from other semantic factors

[Higgins, 2017] **beta-vae: Learning basic visual concepts with a constrained variational framework**, Higgins, I. et al., *ICLR,* 2017

β-VAE

- β-VAE introduces an unintended **trade-off** between **disentanglement** and **reconstruction** accuracy



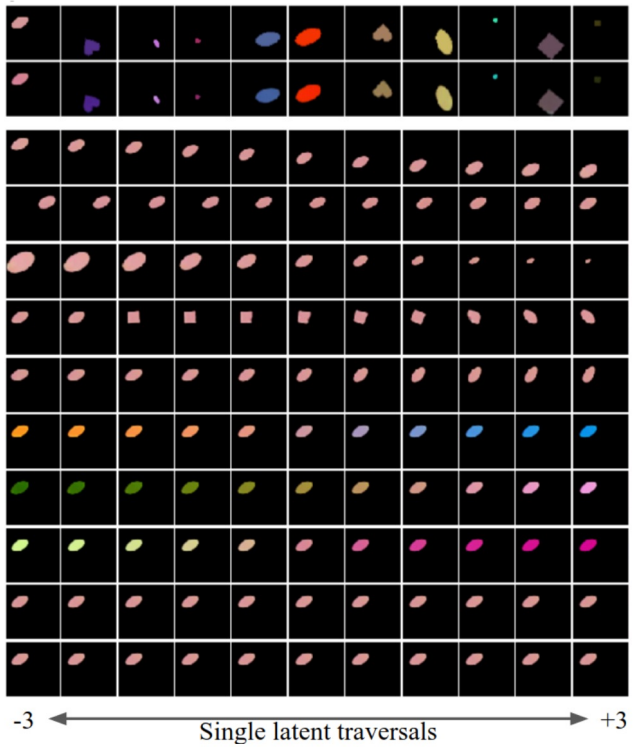
[Burgess, 2018] **Understanding disentangling in beta-vae**, Burgess, C. P. et al., *arXiv,* 2018

AnnealedVAE

- [Burgess, 2018] proposes to **gradually increase the capacity of the model** during training (e.g., $\gamma := 1000$, C increased from 0.5 nats to 25 nats), learning one factor of variation at a time

- The model converges to a local minimum with high disentanglement and good reconstruction

$$\mathcal{L} = \mathbb{E}_{q_\phi(z|x)}[-\log p_\theta(x|z)] + \gamma \cdot \left| KL\left[q_\phi(z|x) \| p(z)\right] - C \right|$$



original

reconstruction

$z_1 \approx y$ position

$z_2 \approx x$ position

$z_3 \approx$ size

$z_4 \approx$ shape

$z_5 \approx$ orientation

$z_6 \approx$ orange → blue

$z_7 \approx$ green → pink

$z_7 \approx$ pale green → red

$KL(q(z_8|x)||p(z_8)) = 0$

$KL(q(z_9|x)||p(z_9)) = 0$

[Burgess, 2018] **Understanding disentangling in beta-vae**, Burgess, C. P. et al., *arXiv*, 2018

Factorial representations

- Several work argue that **factorial representations** i.e., ones with statistically independent latent variables, are **conducive of disentanglement**

- Of course, $p(z) \triangleq \prod_i p(z_i)$ is by design a product of independent factors

- But the inferred prior, a.k.a. **aggregate posterior** $q_\phi(z) \triangleq \sum_n q_\phi(z|x_n)q(n)$, i.e., the distribution of z's to which we map the data, is not necessarily factorial

  - Thus $q_\phi(z)$ is the quantity that these work constrain to be factorial

DIP-VAE

- DIP-VAE [Kumar, 2018] forces the aggregate posterior $q_\phi(z)$ to be close to the factorial prior $p(z) \triangleq \mathcal{N}(0, \mathrm{I}_d)$
  - Specifically, they force the **second moment** to match: $\mathrm{Cov}_{q_\phi(z)}[z] \approx \mathrm{I}_d$, by minimizing

$$-ELBO(\theta, \phi) + \lambda_{od} \underbrace{\sum_{i \neq j} \left(\mathrm{Cov}_{q_\phi(z)}[z]\right)_{ij}^2}_{\text{Off-diagonal coefficients close to 0}} + \lambda_d \underbrace{\sum_i \left(\left(\mathrm{Cov}_{q_\phi(z)}[z]\right)_{ii} - 1\right)^2}_{\text{Diagonal coefficients close to 1}}$$

- By the law of total covariance, $\mathrm{Cov}_{q_\phi(z)}[z]$ can be expressed as a sum of readily computable terms:

$$\mathrm{Cov}_{q_\phi(z)}[z] = \mathbb{E}_{p(x)}\left[\mathrm{Cov}_{q_\phi(z|x)}[z]\right] + \mathrm{Cov}_{p(x)}\left(\mathbb{E}_{q_\phi(z|x)}[z]\right)$$

[Kumar, 2018] **Variational inference of disentangled latent concepts from unlabeled observations**, Kumar, A. et al., *ICLR,* 2018

β-TCVAE and FactorVAE

- FactorVAE [Kim, 2018] and β-TCVAE [Chen, 2018] directly force the aggregate posterior $q_\phi(z)$ to be factorial
  - Write $\bar{q}_\phi(z) \triangleq \prod_i q_\phi(z_i)$ the product of the marginals of the aggregate posterior; minimize:

$$-ELBO(\theta, \phi) + \beta \cdot \underbrace{KL\left(q_\phi(z) || \bar{q}_\phi(z)\right)}_{\text{Penalize non-factorial aggregate posterior}}$$

- This second term is called the **Total Correlation** (TC); it is non-trivial to compute, and the two concurrent work propose two different strategies
  - β-TCVAE uses a biased minibatch estimate, FactorVAE rewrites this term as a density ratio and estimates it by training an auxiliary classifier (likelihood ratio trick)

[Kim, 2018] **Disentangling by factorizing**, Kim, H. et Mnih, A., *ICML*, *2018*
[Chen, 2018] **Isolating sources of disentanglement in variational autoencoders**, Chen, R. T. et al., *NeuRIPS,* 2018

β-TCVAE and FactorVAE



Gender-related latent dimension z_i →

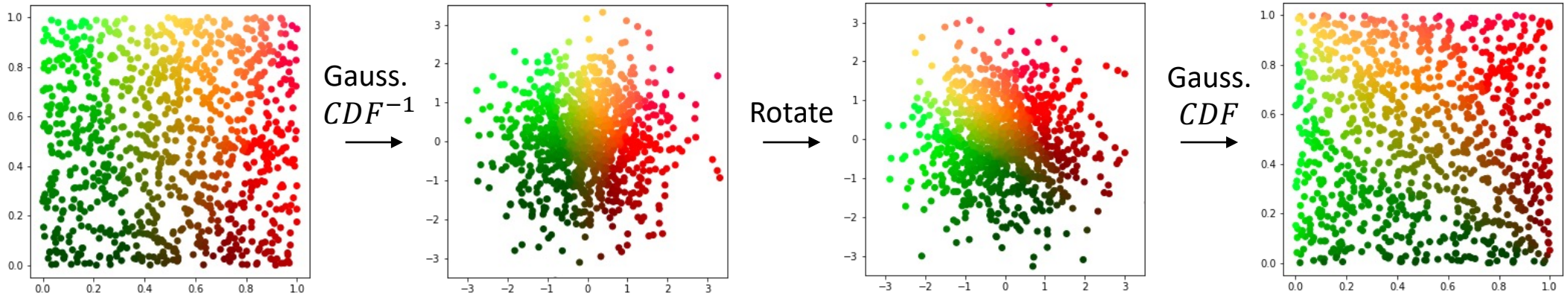
[Chen, 2018] **Isolating sources of disentanglement in variational autoencoders**, Chen, R. T. et al., *NeuRIPS,* 2018

A negative result on unsupervised disentanglement

- Marginal likelihood maximization $p(x) \triangleq \int p_\theta(x|z)p(z)dz$ alone **cannot achieve disentanglement**

- Starting from a disentangled model, with latent variables $z \sim p(z)$ and likelihood $p_\theta(x|z)$; where $p(z) \triangleq \prod_i p_i(z_i)$ factorizes over independent latents z_i, we can
  1. build entangled latent variables $\tilde{z} \triangleq \psi(z)$ by applying a bijective transformation ψ such that:
  2. the entangled $\tilde{z} \sim p(\tilde{z})$ has the same distribution as z, then
  3. absorb the inverse transformation ψ^{-1} in the likelihood $p_\theta(x|\tilde{z})$

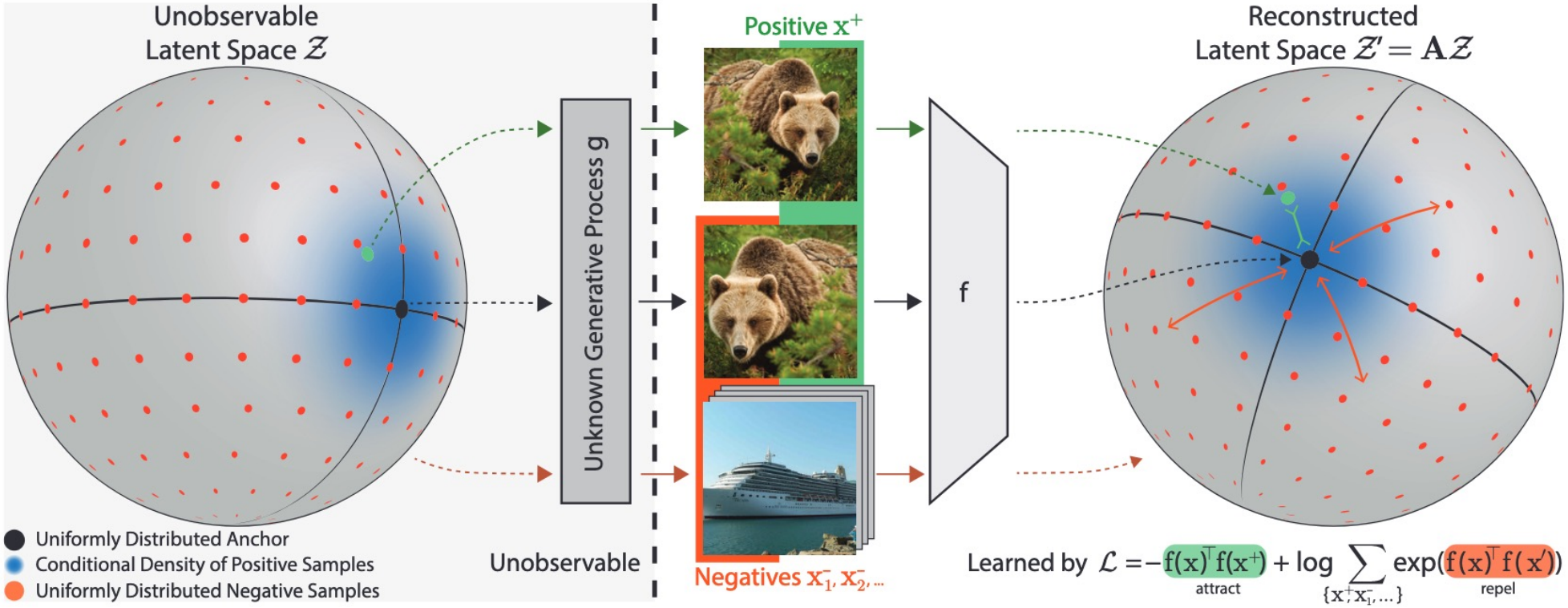
  so that the generative model $\tilde{z} \sim p(\tilde{z}),\ x \sim p_\theta(x|\tilde{z})$ has the same marginal likelihood as $z \sim p(z)$, $x \sim p_\theta(x|z)$ but is completely entangled



[Locatello, 2019] **Challenging common assumptions in the unsupervised learning of disentangled representations**, Locatello, F. et al., *ICML,* 2019

Contrastive learning and disentanglement

- Links have been drawn between **self-supervised learning** and inverting the true data generating process (finding the true generative factors from x)[2]

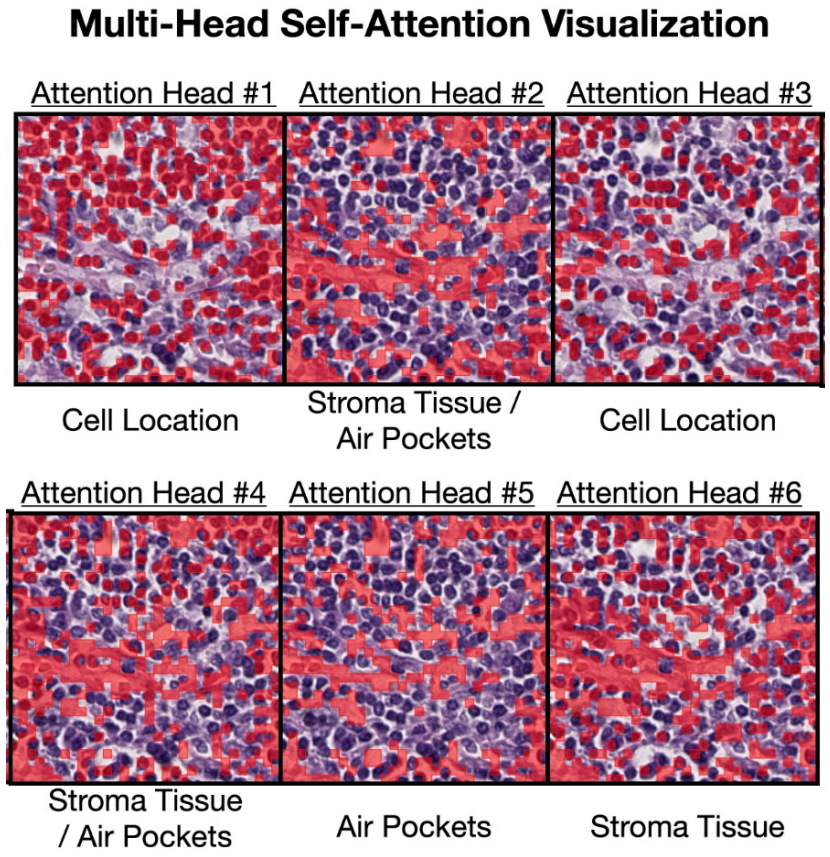


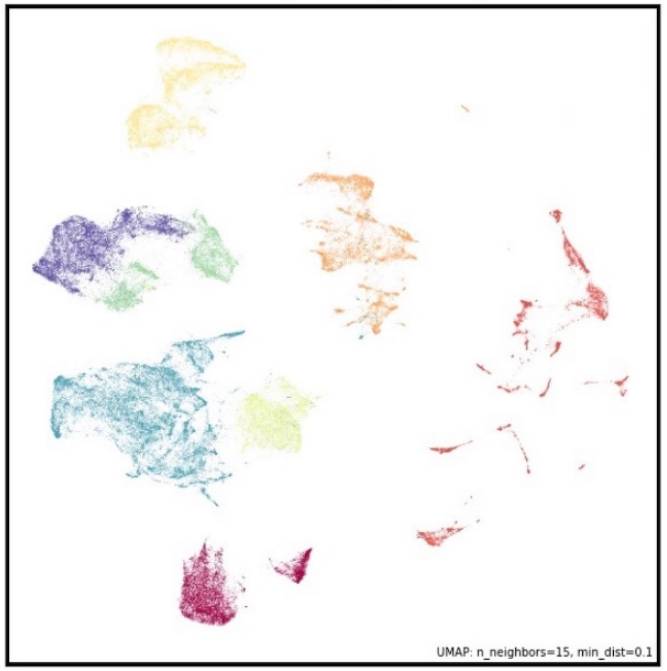
- SSL naturally disentangles in the latent representation z a **content** component (invariant to augmentation) from a **style** component (allowed to change)[1]

[1][Von Kügelgen, 2021] **Self-supervised learning with data augmentations provably isolates content from style**, Von Kügelgen, J. et al., *NeuRIPS,* 2021
[2][Zimmermann, 2021] **Contrastive learning inverts the data generating process**, Zimmermann, R.S. et al., *ICML,* 2021

DINO and disentanglement

- DINO representations seem to possess some disentanglement qualities; here an illustration on histopathology data



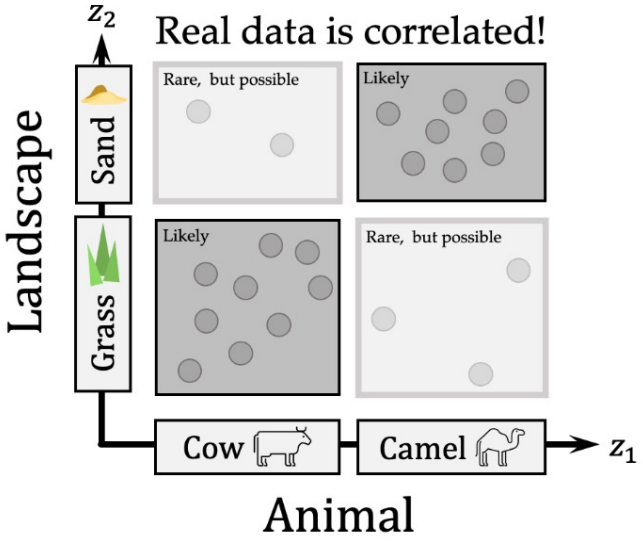


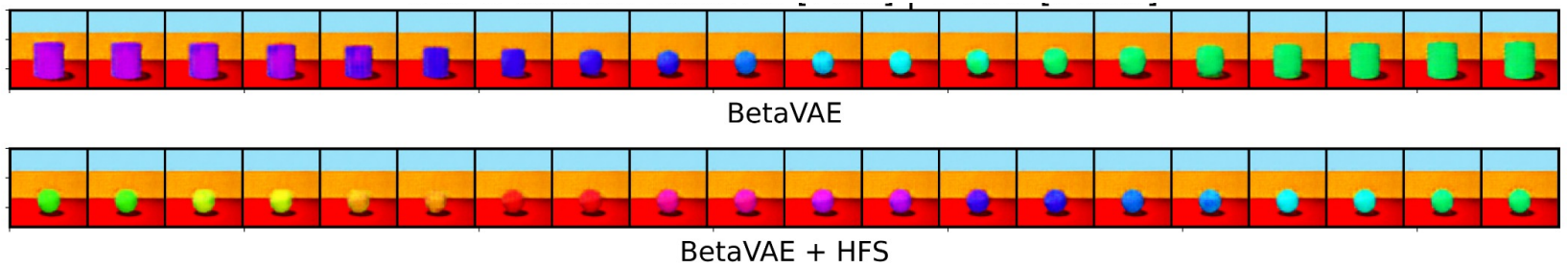
UMAP projection of DINO embeddings naturally clusters different tissue types (shown in different colors)

[Chen, 2022] **Self-Supervised Vision Transformers Learn Visual Concepts in Histopathology**, Chen, R.J. et Krishnan, R.G., *arXiv:2203.00585,* 2022

Hausdorff factorized support

- **Factorized support** can be imposed [Roth, 2023] on $q_\phi(z)$
  - This relaxes the assumption that $q_\phi(z)$ should factorize, which does not hold as well when correlations between attributes exist





[Roth, 2023] **Disentanglement of Correlated Factors via Hausdorff Factorized Support**, Roth, K. et al., *ICLR,* 2023

From disentanglement to identifiability

- Disentanglement relates to the problem of **identifiability** of the model $p_\theta(x|z)$ and of the latent variables z
  - The negative result for marginal likelihood-based training is a non-identifiability result
  - Identifiability up to permutations and signs of the latent variables is the strongest we can hope for

- The problem of identifiability is often discussed from the point of view of **nonlinear ICA**[1] (to which VAEs relate)

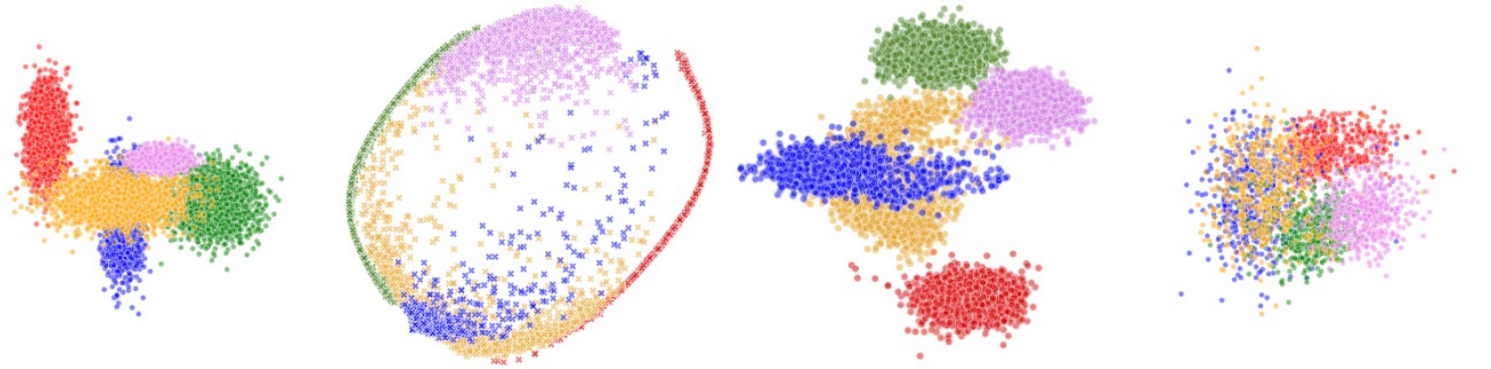
$$x = f(z) \qquad (+\epsilon), \qquad p(z) \triangleq \prod_i p_i(z_i)$$

[1][Hyvärinen, 2023] **Nonlinear independent component analysis for principled disentanglement in unsupervised deep learning**, Hyvärinen, A. et al., *Patterns,* 2023

Conditionally factorial priors

- We can recover a form of identifiability [Khemakhem, 2020] if we observe auxiliary information u, conditionally on which the latent variables z have a factorial prior $p_\theta(z|u)$

- u can be the time index in a time series, a noisy class label, etc.

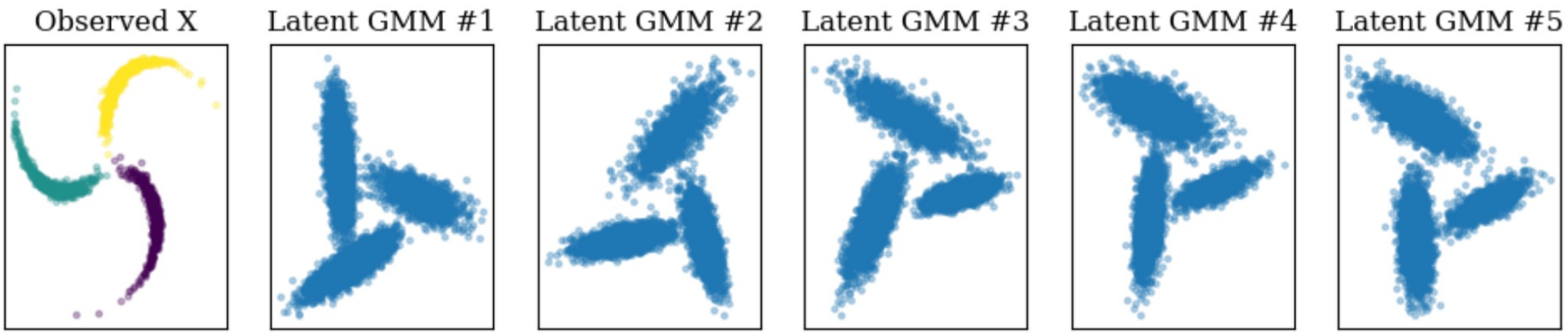


(a) $p_{\boldsymbol{\theta}*}(\mathbf{z}|\mathbf{u})$ (b) $p_{\boldsymbol{\theta}*}(\mathbf{x}|\mathbf{u})$ (c) $p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x},\mathbf{u})$ (d) $p_{\mathrm{VAE}}(\mathbf{z}|\mathbf{x})$

[Khemakhem, 2020] **Variational autoencoders and nonlinear ica: A unifying framework**, Khemakhem, I. et al., *AIStats,* 2020

MoG priors

- It is actually not crucial [Kivva, 2022] for the auxiliary information u to be observed
- For instance if $p_\theta(z)$ is a **mixture of Gaussian** with some technical conditions on the covariances, we can identify $p_\theta(z)$ and the nonlinear mapping f up to permutation, scaling, and translation

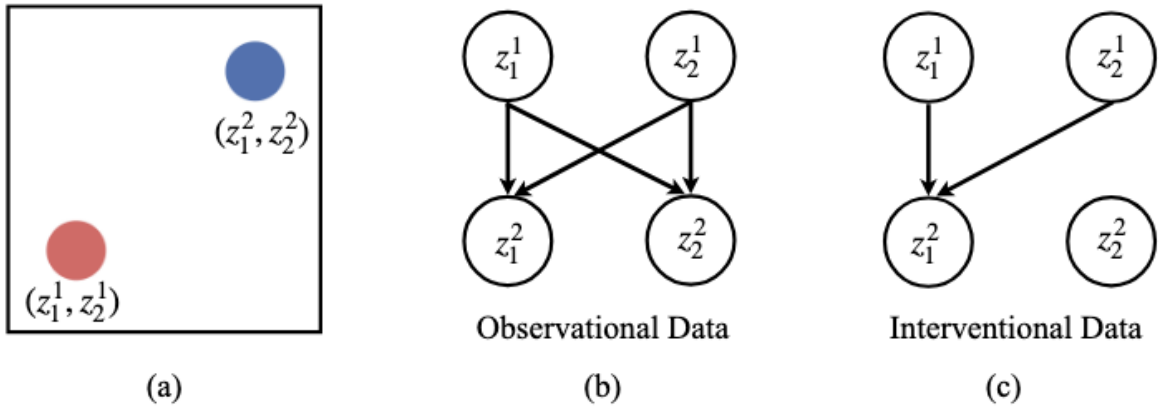


- Under strong assumptions on the model, that f is a piecewise affine function such as MLP w/ ReLU

[Kivva, 2022] **Identifiability of deep generative models without auxiliary information**, Kivva, B. et al., *NeuRIPS,* 2022

Interventional causal representation learning

- Causality is also being investigated extensively as a means for disentanglement
- Identifiability can sometimes be guaranteed [Ahuja, 2023] up to permutation and scaling when **interventional data** rather than observational data is available
  - Data is observed after intervening on a specific factor of variation z_i



(a) (b) (c)

- Breaking correlations between factors allows the "signature" of individual factors to appear in the data

[Ahuja, 2023] **Interventional causal representation learning**, Ahuja, K. et al., *ICML,* 2023

Metrics of disentanglement

- How do we **measure** the quality / **degree of disentanglement** of a representation?
- Metrics assume that the true interpretable **generative factors** $v_1 \cdots v_K$ (or a subset) are **known**, and measure the discrepancy between these factors and the latent variables
  - This limits their use to synthetic datasets with known factors ('dSprites', '3D Faces', '3D Shapes', 'Morpho-MNIST', 'SmallNORB');
  - Or to datasets for which attributes are available, such as 'CelebA' (40 attributes over 202,599 face images!)



- ✓ Shape: square, ellipse, heart
- ✓ Scale: 6 values in $[0.5,1]$
- ✓ Orientation: 40 values in $[0,2\pi]$
- ✓ Position X: 32 values in $[0,1]$
- ✓ Position Y: 32 values in $[0,1]$

Source: dSprites dataset github

Higgins' metric (a.k.a. β-VAE metric)

The **Higgins' metric** is defined as the **accuracy** of a **linear classifier** at **identifying one fixed ground truth factor** given the value of the latent variables, as the other factors vary

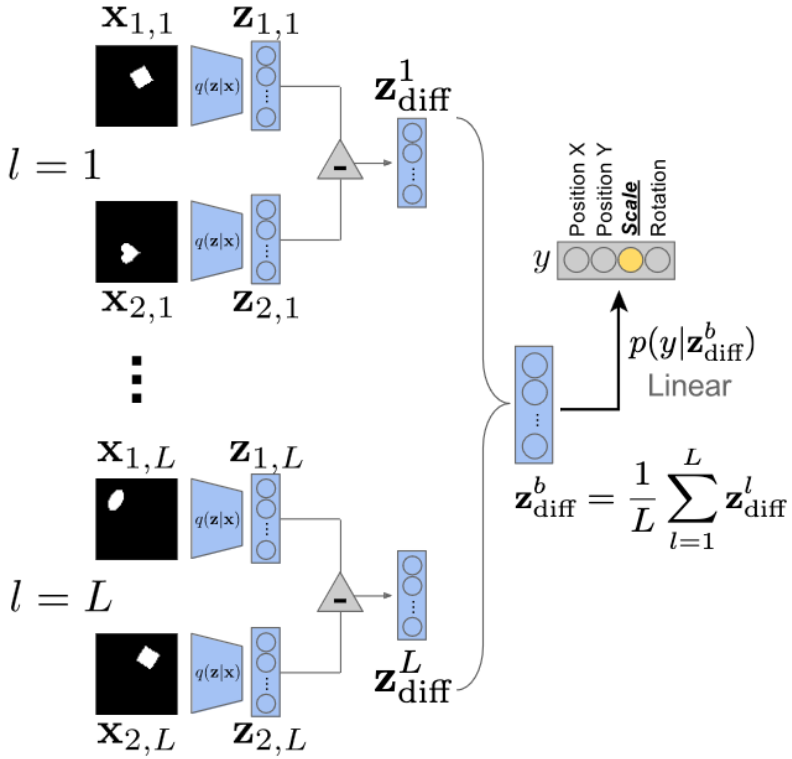


Figure 5: Schematic of the proposed disentanglement metric: over a batch of L samples, each pair of images has a fixed value for one target generative factor y (here $y = scale$) and differs on all others. A linear classifier is then trained to identify the target factor using the average pairwise difference $\mathbf{z}^b_{\text{diff}}$ in the latent space over L samples.

[Higgins, 2017] **beta-vae: Learning basic visual concepts with a constrained variational framework**, Higgins, I. et al., *ICLR,* 2017

Higgins' metric (a.k.a. β-VAE metric)

1. Create a training dataset for a linear classifier
   For $b = 1 \dots B$:

   i. Choose a factor $k_b \sim \mathcal{U}(\{1, \cdots, K\})$ (e.g., $k_b =$ 'scale')

   ii. For a batch of $l = 1 \cdots L$ samples:
      a. Sample a fixed value $v_{k_b}(l)$ for the chosen generative factor uniformly

      b. Sample two data points x_l^1, x_l^2 uniformly among data points with value $v_{k_b}(l)$ of the generative factor

      c. Sample the corresponding latents $z_l^1 \sim q(z|x_l^1)$ and $z_l^2 \sim q(z|x_l^2)$

      d. Compute $z_{\text{diff}}^l \triangleq \left| z_l^1 - z_l^2 \right|$

   iii. Compute $z_{\text{diff}}^b \triangleq \frac{1}{L} \sum_l z_{\text{diff}}^l$

[Higgins, 2017] **beta-vae: Learning basic visual concepts with a constrained variational framework**, Higgins, I. et al., *ICLR,* 2017

Higgins' metric (a.k.a. β-VAE metric)

2. Train the linear classifier to predict the chosen factor k_b from z^b_{diff}, for all $b = 1 \ldots B$

3. The Higgins' metric is the accuracy of this classifier on a test dataset

[Higgins, 2017] **beta-vae: Learning basic visual concepts with a constrained variational framework**, Higgins, I. et al., *ICLR,* 2017

Higgins' metric (a.k.a. β-VAE metric)

- If any given latent is sensitive to only one factor, one of the dimensions of z^b_{diff} will be 0, and it will be easy to predict the fixed factor:
  i.e., **high disentanglement → high Higgins' score**

- If latents are sensitive to multiple factors, no dimension of z^b_{diff} will stand out and it will be difficult to predict the fixed factor:
  i.e., **low disentanglement → low Higgins' score**

- On the other hand, if a generative factor is split over several latents, it does not prevent good prediction
  - i.e. the Higgins' score does **not favor compact representations** that use exactly 1 latent per generative factor

[Higgins, 2017] **beta-vae: Learning basic visual concepts with a constrained variational framework**, Higgins, I. et al., *ICLR,* 2017

FactorVAE metric, SAP score, DCI metric

- The **FactorVAE metric** is another **classifier-based metric** with a parameter-free classifier
  - The prediction of the fixed factor is done based on the index of the latent with smallest variance and a majority vote

- The **SAP score** [Kumar, 2018] further **penalizes lack of compactness**

- The **DCI metric** [Eastwood, 2018] considers the Disentanglement, Completeness (= compactness) and Informativeness (penalizes latent representations that fail to capture some factors)

[Kim, 2018] **Disentangling by factorizing**, Kim, H. et Mnih, A., *ICML*, *2018*

[Kumar, 2018] **Variational inference of disentangled latent concepts from unlabeled observations**, Kumar, A. et al., *ICLR,* 2018

[Eastwood, 2018] **A framework for the quantitative evaluation of disentangled representations**, Eastwood, C. et Williams, C. K., *ICLR,* 2018

Mutual Information Gap (MIG)

- The **MIG** [Chen, 2018] measures for each factor v_k the normalized **gap in mutual information** for the two most predictive latent variables
  - It then averages over the K factors

$$\text{MIG} \triangleq \frac{1}{K}\sum_{k=1}^{K}\frac{1}{H(v_k)}\left(I\left(z_{j^{(k)}};v_k\right)-\max_{j\neq j^{(k)}} I(z_j;v_k)\right)$$

$$\text{where } j^{(k)} \triangleq \operatorname*{argmax}_j I(z_j;v_k)$$

- If each latent variable is sensitive to a single factor → high MIG (close to 1)
  - Unless two latent variables are sensitive to the same factor (MIG close to 0) → **MIG penalizes lack of compactness**

[Chen, 2018] **Isolating sources of disentanglement in variational autoencoders**, Chen, R. T. et al., *NeuRIPS,* 2018

Summary

- Variational auto-encoders are an **approach to representation learning** based on learning a generative model of data
  - They learn **useful, low-dimensional** representations of data

- Ensuring that the true, interpretable generative factors are recovered unambiguously is an open problem

- Sophisticated hierarchical variants of VAEs can **generate high-quality images**, on par with GANs

- **Diffusion models**, **rectified flows** and **language models** represent the state-of-the-art in image synthesis nowadays, are computationally more demanding but simpler to train
  - In that context, variational / quantized autoencoders are mostly used for **perceptual encoding**